投研数据MCP Server系统 — 开发需求说明书

版本: v1.9（终版） **日期:** 2026年3月28日 **文档性质:** 技术需求说明书（面向开发者） **修订说明:** v1.1-v1.8(121项修复+42源精选) → v1.9(6角色终审13项+Research适配11项+终极审查14项=159项)

目录

1. 项目背景与目标
2. 你需要做什么（一句话）
3. 系统架构总览
4. 技术栈要求
5. MCP协议简介（开发者必读）
6. Claude连接方式说明（重要）
7. 8个MCP Server详细需求
8. 核心设计要求
9. 部署方案
10. 开发阶段划分与里程碑
11. 测试策略
12. 验收标准
13. 参考资源
14. 沟通与交付流程
15. Claude调用指南（委托人使用须知） 附录A：各市场精选数据源清单（42个） 附录B：三套投研框架结构与工具映射 附录C：金融术语表

开发者阅读指南（先读这里）

```
第一天（30分钟了解全貌）：
  1. 读第1节"目的概述"——理解这个系统为什么存在
  2. 读第3节"架构总览"——理解shared/adapters + 8个Server的代码结构
  3. 读第5.4节"最小可运行示例"——跑通第一个MCP工具
  4. 翻阅附录B——理解每个工具在投研框架中的位置

第二天（开始编码）：
  1. 先开发shared/公共层（adapters/cache/fallback/config）——这是整个系统的地基
```

```
  2. 读第7节中你当前要开发的Server的工具清单
  3. 读第8节"核心设计要求"——缓存/故障切换/返回格式的代码规范
  4. 读附录A中对应市场的数据源清单——注册API Key

部署时:
  1. 读第9节"部署方案"（含首次部署step-by-step清单）
  2. 读第15节"Claude调用指南"——连接方式+提示词模板

金融术语不确定时:
  1. 查阅附录C术语表
```

---

1. 项目背景与目标

1.0 项目目的概述

本系统的本质不是一个数据工具，而是一套**危机投资哲学的工程化实现**。

委托人拥有一套完整的危机投资体系——核心公式为「超额收益 = 发生可逆危机的好公司 × 被情绪错杀的极低价格 × 市场空间极大的增长潜力 × 耐心等待时间的平方」。围绕这个公式，委托人建立了三套标准化投研框架：成长性公司危机投资投研法（14节）、成熟公司危机投资投研法（8大步骤含评分体系）、Web3项目危机投资投研法（15节），分别覆盖美股、港股、A股和加密货币四个市场。三套框架虽然结构不同，但共享同一个底层逻辑：在可逆危机中找到被情绪错杀的好标的，用10种以上估值方法交叉验证其内在价值，在低于5折时买入，耐心等待价值回归。

这套体系的执行瓶颈在于**数据获取的摩擦**。每个标的的完整投研需要覆盖财务三表、分析师预测、内部人交易、股东结构、行业TAM、竞对对比、10+种估值交叉验证等数十个数据维度。纯人工收集这些数据需要数小时甚至数天，无法支撑每天2个标的的投研节奏。更关键的是，人在收集数据的过程中会逐渐形成偏见——搜索到的第一条新闻会锚定判断方向，而这恰恰违背了委托人"去人欲、守大道"的投资纪律。

本系统要解决的问题是：**把数据获取环节从手动搜索变成自动查询，让委托人把100%的注意力放在判断上，而不是浪费在数据收集上。** 具体而言，通过MCP协议将42个精选金融数据源整合为8个标准化服务：

```
常规对话模式: 约60-90秒数据获取 + 2-3分钟分析 = 3-5分钟完成投研
Research模式(Opus 4.6): 5-15分钟自主深度研究（MCP数据+web搜索+交叉验证） = 更长但更深入
```

最终按照委托人的投研框架输出完整的投委会备忘录。

这个系统不做投资判断——它只做数据的标准化获取和格式化呈现。公式中每个变量的数据输入由系统解决，但公式本身的运算——判断力、耐心和纪律——始终由委托人掌控。当危机出现时（比如一家优

秀公司因财报miss而暴跌69%），委托人能在同一天内完成从"发现机会"到"输出投研报告"的全流程，不错过任何一个被情绪错杀的好标的。

1.1 背景

我是一名投资研究者，使用Anthropic的Claude AI（claude.ai和Claude Desktop）进行投资研究分析。我有三套标准化的投研框架（成长性公司14节/成熟公司8步/Web3项目15节），覆盖美股、港股、A股和加密货币四个市场。

目前的痛点是：Claude每次做投研需要通过网络搜索逐个获取数据，效率低、数据不完整、格式不统一。我需要一个数据中间层，让Claude能够通过MCP协议直接调用结构化的金融数据。

1.2 目标

构建一组MCP Server（Model Context Protocol服务），作为Claude与金融数据源之间的桥梁。

最终效果：我在Claude中输入一个股票代码和指令，Claude通过MCP自动获取该标的的全部结构化数据（财务报表、股价、分析师评级、股东结构、行业数据等），然后结合联网搜索的定性信息，在5-15分钟内（Research模式）生成完整的投研报告。

1.3 使用频率

每天分析2个投资标的，每个标的需要调用约10-20个MCP工具。

---

2. 你需要做什么（一句话）

开发并部署8个MCP Server，整合约25个免费金融数据API/Python库（4个市场精选42个源，去重后约25个需要写adapter），对外暴露约90个标准化的MCP工具（Tool），让Claude能够通过MCP协议直接查询美股、港股、A股和加密货币的结构化金融数据。

---

3. 系统架构总览

```
┌──────────────────────────────────────────────────────────────┐
│                Claude (Desktop App 或 claude.ai)              │
│                                                               │
│  用户输入: "投研指令: BTGO (美股), 执行14节框架"               │
│                                                               │
│  Claude 自动调用:                                             │
│    ├── MCP工具（结构化数据） ← 你要开发的部分                  │
│    ├── Web Search（新闻/研报等非结构化信息） ← Claude自带      │
│    └── 用户上传文件（付费数据源导出） ← 用户手动               │
└──────────────┬────────────────────────────────────────────────┘
```

```
                | MCP协议（Streamable HTTP 或 stdio）
                ↓
┌──────────────────────────────────────────────────────────────────┐
│              云服务器（阿里云/AWS）                                  │
│                                                                   │
│  docker-compose.yml 管理以下容器:                                   │
│                                                                   │
│  ┌──────────────────┐  ┌──────────────────┐                      │
│  │ us-stocks-mcp    │  │ cn-stocks-mcp    │                      │
│  │ 端口: 9001       │  │ 端口: 9002       │                      │
│  └──────────────────┘  └──────────────────┘                      │
│  ┌──────────────────┐  ┌──────────────────┐                      │
│  │ hk-stocks-mcp    │  │ crypto-mcp       │                      │
│  │ 端口: 9003       │  │ 端口: 9004       │                      │
│  └──────────────────┘  └──────────────────┘                      │
│  ┌──────────────────┐  ┌──────────────────┐                      │
│  │ macro-mcp        │  │ sentiment-mcp    │                      │
│  │ 端口: 9005       │  │ 端口: 9006       │                      │
│  └──────────────────┘  └──────────────────┘                      │
│  ┌──────────────────┐  ┌──────────────────┐                      │
│  │ regulatory-mcp   │  │ analyst-mcp      │                      │
│  │ 端口: 9007       │  │ 端口: 9008       │                      │
│  └──────────────────┘  └──────────────────┘                      │
│                                                                   │
│  ┌──────────────────┐                                             │
│  │ Redis 缓存       │  ← 所有Server共享                            │
│  │ 端口: 6379       │                                             │
│  └──────────────────┘                                             │
│                                                                   │
│  ┌──────────────────┐                                             │
│  │ Nginx反向代理    │  ← HTTPS + 路由 + 认证                       │
│  │ 端口: 443        │                                             │
│  └──────────────────┘                                             │
└──────────────────────────────────────────────────────────────────┘
```

每个MCP Server是一个独立的Python进程，通过FastMCP的streamable-http模式提供HTTP服务。
Nginx作为反向代理统一对外暴露HTTPS入口，按路径分发到不同Server，并负责Bearer Token认证。
Redis作为共享缓存，所有Server共用一个Redis实例。

⚠️ **关键架构决策：共享适配器层**

MCP Server之间不能互相调用工具。 每个Server是独立进程，没有内部MCP通信机制。但`analyst_run_multi_valuation`需要跨越多个Server的数据（财务数据在us-stocks-mcp，TVL数

据在crypto-mcp，利率数据在macro-mcp的底层FRED API）。

解决方案：共享适配器代码 + 共享Redis缓存。

所有Server共享一套数据源适配器代码（adapter），每个Server直接import适配器获取底层API数据，不通过MCP协议调用其他Server。由于共享同一个Redis实例，同一条数据只会从外部API获取一次——如果us-stocks-mcp已缓存BTGO的FMP财务数据，analyst-mcp读缓存即可。

项目代码结构（必须按此组织）：

```
mcp-investment-research/
├── shared/                          ← 所有Server共享的代码（核心）
│   ├── __init__.py                  ← ⚠️ 必须存在，否则Python import失败
│   ├── adapters/                    ← 数据源适配器（每个API一个文件）
│   │   ├── __init__.py
│   │   ├── fmp.py                   ← Financial Modeling Prep
│   │   ├── finnhub.py               ← Finnhub
│   │   ├── yahoo.py                 ← yfinance封装
│   │   ├── akshare_adapter.py       ← AKShare封装
│   │   ├── tushare_adapter.py       ← Tushare封装
│   │   ├── baostock_adapter.py      ← BaoStock封装（含login/logout）
│   │   ├── coingecko.py             ← CoinGecko（含Demo Key认证）
│   │   ├── defillama.py             ← DeFiLlama全套子API
│   │   ├── ccxt_adapter.py          ← CCXT交易所封装
│   │   ├── etherscan.py             ← Etherscan
│   │   ├── fred.py                  ← FRED美联储数据
│   │   ├── sec_edgar.py             ← SEC EDGAR
│   │   ├── snapshot.py              ← Snapshot DAO治理
│   │   └── tokenterminal.py         ← TokenTerminal
│   ├── cache.py                     ← Redis缓存工具（含graceful degradation）
│   ├── config.py                    ← 统一配置管理（读.env）
│   ├── validators.py                ← 数据验证（NaN/异常值处理）
│   ├── formatters.py                ← 输出格式化（货币/时区/数值）
│   ├── quota.py                     ← API额度管理
│   ├── stats.py                     ← 轻量级运行统计（见第9.8节监控）
│   └── fallback.py                  ← 故障切换逻辑（含告警阈值）
│
├── us-stocks-mcp/                   ← 只含工具定义 + Server启动
│   ├── server.py                    ← FastMCP工具注册 + 启动入口
│   ├── tools.py                     ← 工具函数（调用shared/adapters）
│   ├── Dockerfile
│   └── requirements.txt             ← 只含该Server需要的依赖
│
├── cn-stocks-mcp/                   ← 同上结构
│   ├── server.py
│   ├── tools.py
│   ├── Dockerfile
│   └── requirements.txt
│
├── crypto-mcp/
├── hk-stocks-mcp/
```

```
├── macro-mcp/
├── sentiment-mcp/
├── regulatory-mcp/
├── analyst-mcp/                    ← import shared/adapters中的所有适配器
│   ├── server.py
│   ├── tools.py                    ← multi_valuation直接调adapter，不跨Server
│   ├── valuation_engine.py         ← 12种估值方法的计算逻辑
│   ├── Dockerfile
│   └── requirements.txt
│
├── tests/                          ← 测试目录（见第11节）
│   ├── unit/                       ← 单元测试（mock外部API）
│   ├── integration/                ← 集成测试（调用真实API）
│   └── e2e/                        ← 端到端测试（完整投研流程）
│
├── docker-compose.yml
├── .dockerignore                   ← ⚠️ 必须创建（见下方）
├── nginx/
│   └── nginx.conf.template
├── .env
├── .env.example
└── README.md
```

⚠️ **必须创建`.dockerignore`文件：** 防止`COPY`时把.env（含API Key明文）、.git、其他Server代码等复制进容器：

```
# .dockerignore
.git
.env
*.md
logs/
__pycache__/
*.pyc
.pytest_cache/
tests/
```

关键：每个Server的Dockerfile需要同时COPY shared目录和本Server代码（不要用`COPY . .`全量复制）。

完整的Dockerfile模板见第9.1节。核心COPY逻辑示例：

```dockerfile
```

```
FROM python:3.11-slim
WORKDIR /app
ENV PYTHONPATH=/app
COPY us-stocks-mcp/requirements.txt ./requirements.txt
RUN pip install --no-cache-dir -r requirements.txt
COPY shared/ /app/shared/              # ← 共享适配器代码
COPY us-stocks-mcp/*.py /app/          # ← 只复制本Server的代码，不要COPY . .
RUN mkdir -p /app/logs
CMD python server.py --transport streamable-http --port 9001
```

每个Server的Python代码中通过相对导入使用共享适配器：

```python
# us-stocks-mcp/tools.py
from shared.adapters import fmp, finnhub, yahoo
from shared.cache import get_cached_or_fetch
from shared.fallback import get_data_with_fallback
from shared.formatters import format_financial_statement

async def us_get_income_statement(symbol, period="annual", limit=3):
    return await get_data_with_fallback(symbol, [
        ("FMP", lambda s: fmp.get_income_statement(s, period, limit)),
        ("yfinance", lambda s: yahoo.get_income_statement(s, period, limit)),
    ])
```

```python
# analyst-mcp/valuation_engine.py — 直接调adapter，不跨Server
from shared.adapters import fmp, finnhub, coingecko, defillama, fred
from shared.cache import get_cached_or_fetch

async def run_growth_valuation(symbol):
    # 直接调FMP获取财务数据（或从Redis缓存读取）
    financials = await fmp.get_income_statement(symbol)
    price = await fmp.get_quote(symbol)
    risk_free_rate = await fred.get_series("DGS10")  # ← 直接调FRED，不走macro-mcp
    peers = await fmp.get_peers(symbol)
    # ... 计算12种估值方法
```

---

4. 技术栈要求

组件	选型	版本要求	原因
语言	Python	≥ 3.11	MCP SDK + 金融数据库生态最丰富
MCP框架	**FastMCP**	最新版	MCP官方SDK的简化封装，pip install mcp
缓存	Redis	≥ 7.0	高性能TTL缓存
HTTP客户端	httpx	≥ 0.25	异步HTTP请求
容器	Docker + Docker Compose	最新版	统一部署和管理
反向代理	Nginx	最新版	HTTPS + SSE支持 + 认证
数据处理	pandas	≥ 2.0	金融数据处理

⚠️ **重要：全项目统一使用FastMCP框架。不要使用MCP SDK的底层Server类。**

Python金融数据库（必须在requirements.txt中pin版本）：

```
akshare==1.14.86          # A股/港股数据（⚠️ pin版本！接口变动频繁，升级前必须全量测试）
tushare==1.4.8            # A股深度数据
baostock==0.8.8           # A股历史数据
yfinance==0.2.38          # 美股/港股行情（⚠️ 非官方库，Yahoo可能随时变更API导致失效）
ccxt==4.3.0               # 加密货币交易所数据
fredapi==0.5.1            # 美联储经济数据
httpx==0.27.0             # 异步HTTP客户端
redis==5.0.0              # Redis客户端
pandas==2.2.0             # 数据处理
mcp>=1.0.0                # MCP SDK（含FastMCP，FastMCP自带HTTP服务器，不需要额外的
FastAPI/uvicorn）
```

⚠️ **yfinance稳定性警告：** yfinance是非官方库，Yahoo会不定期变更API导致yfinance大面积失效。所有依赖yfinance的工具必须有不依赖yfinance的后备源（如FMP或AKShare）。开发者应关注 https://github.com/ranaroussi/yfinance/issues 获取故障信息。

⚠️ **每个Server只安装自己需要的依赖。** 不要给所有Server安装全部库（8个完整镜像约4.4GB，磁盘吃紧）。以下是各Server的依赖拆分示例：

```
us-stocks-mcp/requirements.txt:
  mcp>=1.0.0, httpx==0.27.0, redis==5.0.0, pandas==2.2.0, yfinance==0.2.38

cn-stocks-mcp/requirements.txt:
  mcp>=1.0.0, httpx==0.27.0, redis==5.0.0, pandas==2.2.0,
  akshare==1.14.86, tushare==1.4.8, baostock==0.8.8

hk-stocks-mcp/requirements.txt:
  mcp>=1.0.0, httpx==0.27.0, redis==5.0.0, pandas==2.2.0, yfinance==0.2.38

crypto-mcp/requirements.txt:
  mcp>=1.0.0, httpx==0.27.0, redis==5.0.0, pandas==2.2.0, ccxt==4.3.0

macro-mcp/requirements.txt:
  mcp>=1.0.0, httpx==0.27.0, redis==5.0.0, pandas==2.2.0, fredapi==0.5.1

sentiment-mcp/requirements.txt:
  mcp>=1.0.0, httpx==0.27.0, redis==5.0.0, pandas==2.2.0

regulatory-mcp/requirements.txt:
  mcp>=1.0.0, httpx==0.27.0, redis==5.0.0, pandas==2.2.0

analyst-mcp/requirements.txt（依赖最多——需要跨市场聚合数据计算估值）:
  mcp>=1.0.0, httpx==0.27.0, redis==5.0.0, pandas==2.2.0,
  fredapi==0.5.1, yfinance==0.2.38
  # 注: analyst-mcp通过httpx直接调用CoinGecko/DeFiLlama REST API，不需要装ccxt
  # 只有需要yfinance做后备数据源和fredapi获取利率才额外安装
```

---

5. MCP协议简介（开发者必读）

5.1 什么是MCP

MCP（Model Context Protocol）是Anthropic定义的一个开放协议，允许AI模型（Claude）调用外部工具和数据源。你可以把它理解为"Claude的插件系统"。

一个MCP Server对外暴露若干"工具"（Tool），每个工具有名称、描述和参数定义。Claude看到这些工具后，会根据用户的提问自动决定调用哪个工具、传什么参数。

5.2 协议交互流程

```
1. Claude连接MCP Server
```

```
2. Claude调用 tools/list → Server返回所有可用工具的名称、描述、参数定义
3. 用户提问后，Claude决定调用某个工具
4. Claude调用 tools/call → Server执行逻辑，返回结果
5. Claude收到结果，整合到回答中
```

5.3 传输方式

MCP支持两种传输方式，**本项目必须同时支持两种**：

Streamable HTTP（用于远程部署/生产环境）：Server作为HTTP服务运行，Claude通过HTTPS连接。 适合部署到云服务器，多设备共享。 需要Nginx支持SSE长连接（见部署方案）。

stdio（用于本地开发调试 + Claude Desktop本地连接）：Server作为子进程运行，通过标准输入/输出通信。 Claude Desktop可直接通过stdio连接本地Server。 开发阶段调试首选模式。

5.4 最小可运行示例

全项目统一使用此模式（**FastMCP**）：

```python
```

```python
"""
最小的MCP Server示例 - 一个能查股价的Server
启动方式1 (stdio): python server.py
启动方式2 (HTTP):  python server.py --transport streamable-http --port 9001
"""
from mcp.server.fastmcp import FastMCP
import yfinance as yf
import asyncio

# 创建MCP Server实例
mcp = FastMCP("demo-stock-server")

# 定义一个工具 —— 注意必须是async函数
@mcp.tool()
async def us_get_stock_price(symbol: str) -> str:
    """获取美股的最新价格和基本行情数据。

    Args:
        symbol: 美股股票代码，如 AAPL, BTGO, COIN
    """
    # ⚠️ yfinance是同步库，必须用asyncio.to_thread包装
    def _fetch():
        ticker = yf.Ticker(symbol)
        return ticker.info

    info = await asyncio.to_thread(_fetch)

    # 返回人类可读的文本格式（不要返回原始JSON）
    return (
        f"股票: {info.get('shortName', symbol)}\n"
        f"最新价: ${info.get('currentPrice', 'N/A')}\n"
        f"市值: ${info.get('marketCap', 'N/A'):,}\n"
        f"PE(TTM): {info.get('trailingPE', 'N/A')}\n"
        f"52周最高: ${info.get('fiftyTwoWeekHigh', 'N/A')}\n"
        f"52周最低: ${info.get('fiftyTwoWeekLow', 'N/A')}"
    )

# 启动Server
if __name__ == "__main__":
    import sys
    if "--transport" in sys.argv:
        # HTTP模式（生产环境）
        mcp.run(transport="streamable-http", port=9001)
```

```
    else:
        # stdio模式（本地调试 / Claude Desktop）
        mcp.run(transport="stdio")
```

⚠️ **健康检查端点的实现方式：**

最终决定：全项目统一使用纯FastMCP写法。 不使用FastAPI+FastMCP共存模式（增加复杂度，MCP SDK版本间兼容性不确定）。

健康检查在Docker层面完成，通过检测TCP端口是否在监听判断Server是否存活，不依赖任何HTTP端点：

```yaml
# docker-compose.yml 中每个Server的healthcheck配置
healthcheck:
  test: ["CMD-SHELL", "python -c \"import socket; s=socket.socket(); s.settimeout(2)
  interval: 30s
  timeout: 10s
  retries: 3
```

注：不使用GET请求检查`/mcp`端点——MCP Streamable HTTP使用POST方法（JSON-RPC），GET请求会返回405导致healthcheck误判失败。

Claude连接这个Server后，当用户问"BTGO的股价是多少"，Claude会自动调用us_get_stock_price("BTGO")。

5.5 关键注意事项

工具名称必须加市场前缀。 因为8个Server同时连接Claude时，如果多个Server都有`get_stock_price`，Claude无法区分调用哪个。命名规则：

```
us-stocks-mcp 的工具:   us_get_stock_price, us_get_financials,
us_get_insider_trades ...
cn-stocks-mcp 的工具:   cn_get_stock_price, cn_get_financials,
cn_get_insider_trades ...
hk-stocks-mcp 的工具:   hk_get_stock_price, hk_get_financials ...
crypto-mcp 的工具:      crypto_get_token_price, crypto_get_protocol_tvl ...
macro-mcp 的工具:       macro_get_us_gdp, macro_get_china_cpi ...
sentiment-mcp 的工具:   sentiment_get_fear_greed, sentiment_get_news ...
regulatory-mcp 的工具:  reg_get_sec_filing, reg_get_cn_announcement ...
analyst-mcp 的工具:     analyst_get_consensus, analyst_run_multi_valuation ...
```

同步库必须用asyncio.to_thread包装。 AKShare、BaoStock、yfinance都是同步库，但MCP Server基于异步框架。直接调用同步函数会阻塞整个事件循环，导致其他请求无法处理。

```python
# ❌ 错误：直接调用同步函数
@mcp.tool()
async def cn_get_stock_price(symbol: str) -> str:
    data = ak.stock_zh_a_hist(symbol=symbol)  # 会阻塞整个Server!
    return format_data(data)

# ✅ 正确：用asyncio.to_thread包装
@mcp.tool()
async def cn_get_stock_price(symbol: str) -> str:
    data = await asyncio.to_thread(ak.stock_zh_a_hist, symbol=symbol)
    return format_data(data)
```

BaoStock需要特殊处理。 每次请求需要独立login/logout：

```python
async def baostock_query(query_func, *args, **kwargs):
    """BaoStock要求每次查询独立登录"""
    def _query():
        bs.login()
        try:
            result = query_func(*args, **kwargs)
            data = result.get_data()
            return data
        finally:
            bs.logout()
    return await asyncio.to_thread(_query)
```

5.6 官方参考

资源	链接
MCP协议规范	https://spec.modelcontextprotocol.io
Python SDK（含FastMCP）	https://github.com/modelcontextprotocol/python-sdk
官方快速入门	https://modelcontextprotocol.io/quickstart/server
官方Server示例	https://github.com/modelcontextprotocol/servers

6. Claude连接方式说明（重要）

本系统的主要使用平台是claude.ai（Research功能+Opus 4.6），而非Claude Desktop。 开发者必须确保HTTP模式的生产可用性——这是claude.ai连接的唯一方式。

6.1 Claude.ai（主要使用平台，Research模式载体）

claude.ai通过Integrations连接自建MCP Server。Research功能只在claude.ai上可用。

连接步骤：

```
1. 打开claude.ai → Settings → Integrations（或Connected Apps / MCP Servers）
2. 添加自定义Server → 输入URL和认证信息
3. 需要添加8次（每个Server一个独立连接）：

   名称: us-stocks    URL: https://your-domain.com/us-stocks/mcp
   名称: cn-stocks    URL: https://your-domain.com/cn-stocks/mcp
   名称: hk-stocks    URL: https://your-domain.com/hk-stocks/mcp
   名称: crypto       URL: https://your-domain.com/crypto/mcp
   名称: macro        URL: https://your-domain.com/macro/mcp
   名称: sentiment    URL: https://your-domain.com/sentiment/mcp
   名称: regulatory   URL: https://your-domain.com/regulatory/mcp
   名称: analyst      URL: https://your-domain.com/analyst/mcp

4. 每个连接都需要设置Bearer Token认证（使用.env中的MCP_AUTH_TOKEN）
5. 连接成功后，在对话中选择Research功能 + Opus 4.6即可自动调用MCP工具

⚠️ claude.ai的Integrations界面可能随版本更新变化。
如果以上步骤不适用，请查阅 https://docs.claude.com 的最新MCP连接指南。
```

6.2 Claude Desktop（本地开发调试+备用）

Claude Desktop用于开发阶段的本地调试（stdio模式直连），以及Research不可用时的备用方案。注意：Claude Desktop不**支持Research功能**，只能用常规对话模式。

stdio模式连接（本地运行）：

编辑配置文件：

- macOS: `~/Library/Application Support/Claude/claude_desktop_config.json`
- Windows: `%APPDATA%\Claude\claude_desktop_config.json`

```
json
```

```
{
  "mcpServers": {
    "us-stocks": {
      "command": "python",
      "args": ["/path/to/us-stocks-mcp/server.py"],
      "env": {
        "FMP_API_KEY": "your_key",
        "FINNHUB_API_KEY": "your_key"
      }
    },
    "cn-stocks": {
      "command": "python",
      "args": ["/path/to/cn-stocks-mcp/server.py"],
      "env": {
        "TUSHARE_TOKEN": "your_token"
      }
    },
    "crypto": {
      "command": "python",
      "args": ["/path/to/crypto-mcp/server.py"],
      "env": {
        "COINGECKO_API_KEY": "your_key"
      }
    }
  }
}
```

Streamable HTTP模式连接（远程服务器）：

```json
{
  "mcpServers": {
    "us-stocks": {
      "url": "https://your-domain.com/us-stocks/mcp",
      "headers": {
        "Authorization": "Bearer YOUR_SECRET_TOKEN"
      }
    }
  }
}
```

6.3 交付要求

开发者需要确保：

1. 每个Server都支持 `--transport stdio` 和 `--transport streamable-http` 两种启动模式
2. 提供完整的Claude Desktop配置文件模板（stdio版和HTTP版各一份）
3. 在README中写明claude.ai连接步骤（HTTP模式）和Claude Desktop连接步骤（stdio+HTTP）
4. **HTTP模式是生产环境的唯一模式——必须确保其稳定性高于stdio模式**

6.4 本地开发环境（不需要Docker）

```
开发者本地编码调试时不需要Docker和Nginx：
  1. Python 3.11+
  2. Redis（本地安装或：docker run -d -p 6379:6379 redis:7-alpine）
  3. pip install -r requirements.txt（当前Server的依赖）
  4. 调试方式：python server.py（默认stdio模式）
  5. Claude Desktop通过stdio直接连接本地Server调试
  6. 不需要Nginx、SSL、域名（本地用stdio不走HTTP）
```

6.5 Claude Desktop MCP连接数注意

```
⚠️ Claude Desktop可能对同时连接的MCP Server数量有限制。
自建8个Server + 已连接的FactSet/Quartr/Aiera等 = 可能达到15+个。

如果遇到连接数超限：
  → 按第15.1节的连接策略，只连接当天分析需要的4个Server
  → regulatory-mcp和sentiment-mcp使用频率最低，优先断开
```

---

7. 8个MCP Server详细需求

⚠️ **注意：以下所有工具名称已加市场前缀，避免跨Server命名冲突。**

7.1 Server #1: us-stocks-mcp（美股数据服务）

优先级: ★★★★★（第一个开发）

工具清单

工具名称	参数	返回内容	主数据源	后备数据
us_get_stock_price	symbol, period(1d/5d/1mo/1y)	最新价/开高低收/成交量/涨跌幅/市值	yfinance	FMP→Alpha Vantage
us_get_price_history	symbol, start_date, end_date, interval(daily/weekly)	历史OHLCV数据表	yfinance	FMP
us_get_income_statement	symbol, period(annual/quarter), limit	利润表: 收入/毛利/营业利润/净利润/EPS	FMP	yfinance Alpha Vantage
us_get_balance_sheet	symbol, period(annual/quarter), limit	资产负债表: 总资产/总负债/股东权益/现金/债务	FMP	yfinance
us_get_cashflow	symbol, period(annual/quarter), limit	现金流量表: 经营/投资/筹资活动现金流	FMP	yfinance
us_get_ratios	symbol	PE/PB/PS/EV-EBITDA/ROE/ROA/ROIC/毛利率/净利率/负债率/流动比率/速动比率	FMP	基于财报行计算
us_get_analyst_ratings	symbol	评级分布(买入/持有/卖出)+目标价(高/低/均值)+EPS预测	Finnhub	FMP
us_get_insider_trades	symbol, limit	内部人交易记录: 姓名/职位/交易日期/买卖方向/数量/价格/金额	Finnhub	FMP
us_get_institutional_holders	symbol	机构持仓: 机构名称/持股数/占比/变动(增持/减持/新建仓)	FMP	yfinance
us_get_company_profile	symbol	公司简介/行业/员工数/成立日期/CEO/网站/交易所	FMP	Finnhub

工具名称	参数	返回内容	主数据源	后备数据
us_get_sec_filings	symbol, filing_type(10-K/10-Q/8-K), limit	SEC文件列表: 类型/日期/链接	SEC EDGAR API	Finnhul
us_get_short_interest	symbol	做空股数/做空比例/空头回补天数/做空变动	Finnhub	FMP
us_get_earnings_calendar	symbol	下次财报日期/上次EPS(实际vs预期)	Finnhub	FMP
us_get_peers	symbol	同行业可比公司列表+各自PE/PB/市值/收入增速	FMP	Finnhul
us_get_news	symbol, limit	最近新闻: 标题/来源/日期/摘要/链接	Finnhub	Marketa
us_get_crisis_signals	symbol	危机检测数据包: 裁员/重组新闻+SG&A趋势+最近融资/发债+信用评级+现金/短期负债比+资产出售	Finnhub(新闻)+FMP(财务)+SEC EDGAR(8-K)	yfinanc

us_get_crisis_signals说明： 服务于成熟公司框架"检测危机"三步法（自救/断臂/他救）。一次性返回判断危机可逆性所需的全部数据。

数据源API注册信息

数据源	注册地址	API Key获取	免费额度	稳定性
FMP	https://financialmodelingprep.com/register	注册即得	250次/天	⭐⭐⭐⭐⭐ 稳定
Finnhub	https://finnhub.io/register	注册即得	60次/分钟	⭐⭐⭐⭐⭐ 稳定
yfinance	无需注册	pip install yfinance	无限	⚠️ ⭐⭐⭐ 非官方库，可能突然失效
Alpha Vantage	https://www.alphavantage.co/support/#api-key	注册即得	25次/天	⭐⭐⭐⭐ 稳定但额度极少
SEC EDGAR	https://www.sec.gov/os/accessing-edgar-data	设置User-Agent头	10次/秒	⭐⭐⭐⭐⭐ 政府API极稳定

数据源	注册地址	API Key获取	免费额度	稳定性
Marketaux	https://www.marketaux.com/register	注册即得	100次/天	⭐⭐⭐⭐ 稳定

7.2 Server #2: cn-stocks-mcp（A股数据服务）

优先级: ★★★★★（第一批开发）

工具清单

工具名称	参数	返回内容	主数据源	后备数据源
cn_get_stock_price	symbol(如000001/600519)	最新价/开高低收/成交量/换手率/市值/PE/PB	AKShare	Tushare
cn_get_price_history	symbol, start_date, end_date, adjust(qfq/hfq/不复权)	历史OHLCV数据表	AKShare	BaoStock
cn_get_income_statement	symbol, period(annual/quarter)	利润表: 营收/营业利润/净利润/EPS/扣非净利润	Tushare	AKShare
cn_get_balance_sheet	symbol, period(annual/quarter)	资产负债表: 总资产/总负债/股东权益/现金/有息负债	Tushare	AKShare
cn_get_cashflow	symbol, period(annual/quarter)	现金流量表: 经营/投资/筹资现金流	Tushare	AKShare
cn_get_ratios	symbol	PE/PB/PS/ROE/ROA/毛利率/净利率/资产负债率/流动比率	Tushare	BaoStock
cn_get_shareholders	symbol	前十大股东+持股比例+变动+股东户数及变化趋势	Tushare	AKShare

工具名称	参数	返回内容	主数据源	后备数据源
cn_get_insider_trades	symbol	高管增减持:姓名/职位/日期/方向/数量/价格	Tushare	AKShare
cn_get_analyst_ratings	symbol	卖方评级分布+一致预期EPS/营收+目标价	Tushare	AKShare
cn_get_northbound_flow	symbol或market(沪股通/深股通)	北向资金:今日净买入/累计持仓/持仓市值占比	AKShare	Tushare
cn_get_margin_data	symbol	融资融券:融资余额/融券余额/融资买入额/融券卖出量	AKShare	Tushare
cn_get_dragon_tiger	symbol或date	龙虎榜:上榜原因/买入前五/卖出前五/机构参与	AKShare	Tushare
cn_get_block_trades	symbol, limit	大宗交易:日期/价格/溢折价率/买方/卖方营业部	AKShare	Tushare
cn_get_pledge_data	symbol	股权质押:质押股数/占总股本比例/质押笔数	Tushare	AKShare
cn_get_industry_pe	industry_name	行业PE/PB中位数+分位数+历史趋势	Tushare	AKShare
cn_get_crisis_signals	symbol	危机检测数据包:裁员新闻+管理费用趋势+最近定增/发债+现金/短期负债比+资产出售公告	AKShare(新闻+公告)+Tushare(财务)	—

cn_get_crisis_signals说明： 与美股版功能相同，服务于成熟公司框架"检测危机"。数据源替换为A股生态。

数据源信息

数据源	安装	认证	免费额度	注意事项
AKShare	pip install akshare==1.14.86	无需认证	无限	⚠️ 函数名/参数经常变更，必须pin版本
Tushare Pro	pip install tushare==1.4.8	注册token	按积分（见下方说明）	需Pro会员才能访问核心接口
BaoStock	pip install baostock==0.8.8	无需认证	无限	每次查询需login/logout

⚠️ Tushare积分体系说明（重要）

Tushare Pro不是简单的"每天200-2000次"。它有积分体系，不同接口需要不同积分等级：

```
积分获取方式：
  - 注册：100分
  - 邮箱验证：+50分
  - 手机验证：+100分
  - 充值¥500（Pro会员）：+2000分（推荐）

核心接口所需积分：
  - daily（日线行情）：120分 → 免费账号可用
  - income/balancesheet/cashflow（财务三表）：500分 → 需Pro
  - forecast（业绩预告）：500分 → 需Pro
  - express（业绩快报）：500分 → 需Pro
  - top10_holders（前十大股东）：600分 → 需Pro
  - stk_holdernumber（股东户数）：600分 → 需Pro
  - adj_factor（复权因子）：500分 → 需Pro
  - consensus（分析师一致预期）：1800分 → 需Pro + 社区贡献积分

建议：直接充值¥500/年Pro会员获取2000分。
仍有少数接口(如consensus)需更高积分，用AKShare作为后备。
```

A股代码格式说明

A股股票代码在不同数据源中格式不同，Server内部必须自动转换：

```python
```

```python
def convert_symbol(symbol: str, target: str) -> str:
    """
    统一代码格式转换
    用户输入: "000001" 或 "600519" (纯数字, 6位)

    target格式:
      akshare:  "000001"
      tushare:  "000001.SZ" 或 "600519.SH"
      baostock: "sz.000001" 或 "sh.600519"
      yfinance: "000001.SZ" 或 "600519.SS"

    规则:
      - 0/3开头 → 深圳 (SZ/sz)
      - 6开头 → 上海 (SH/sh)
      - 8/4开头 → 北交所 (BJ/bj)
    """
```

7.3 Server #3: hk-stocks-mcp（港股数据服务）

优先级: ★★★★（第二批开发）

工具清单

工具名称	参数	返回内容	主数据源	后备数据源
hk_get_stock_price	symbol(如9988/0700)	最新价/成交量/市值/PE/PB/股息率	yfinance(.HK)	AKShare
hk_get_price_history	symbol, start_date, end_date	历史OHLCV	yfinance	AKShare
hk_get_income_statement	symbol, period	利润表	yfinance	AKShare
hk_get_balance_sheet	symbol, period	资产负债表	yfinance	AKShare
hk_get_cashflow	symbol, period	现金流量表	yfinance	AKShare

工具名称	参数	返回内容	主数据源	后备数据源
hk_get_ratios	symbol	PE/PB/ROE/股息率/净利率等	yfinance	AKShare
hk_get_southbound_flow	symbol或date	南向资金:每日净买入/累计持仓	AKShare	Tushare
hk_get_connect_holdings	symbol	港股通持仓:持股数量/占比/变动	AKShare	Tushare
hk_get_analyst_ratings	symbol	分析师评级+目标价	yfinance	AKShare
hk_get_company_profile	symbol	公司简介/行业/员工数	yfinance	AKShare
hk_get_crisis_signals	symbol	危机检测数据包:裁员/重组新闻+管理费用趋势+最近融资/发债+现金/短期负债比	yfinance(新闻)+AKShare(财务)+港交所披露易(公告)	—

hk_get_crisis_signals说明： 与美股/A股版功能相同，服务于成熟公司框架"检测危机"。港股投研中的必备工具——分析9988(阿里)、0700(腾讯)等标的的危机可逆性。

港股代码格式

```python
def convert_hk_symbol(symbol: str, target: str) -> str:
    """
    标准化港股代码为4-5位:
      "00700" → "0700.HK" (去掉多余前导零到4位)
      "700"   → "0700.HK" (不足4位时左补零)
      "9988"  → "9988.HK" (正常4位不处理)
      "2800"  → "2800.HK" (ETF等正常处理)

    规则: 先去掉全部前导零，再左补零到4位（最少4位，最多5位如01810）
    yfinance格式: "0700.HK"
    """
```

7.4 Server #4: crypto-mcp（加密货币数据服务）

优先级: ★★★★★（第一批开发）

工具清单

工具名称	参数	返回内容	主数据源
crypto_get_token_price	symbol(如BTC/ETH/ASTER)	最新价/24h涨跌/市值/交易量/FDV	CoinGecko
crypto_get_price_history	symbol, days(7/30/90/365/max)	历史价格数据	CoinGecko
crypto_get_market_data	symbol	市值/FDV/流通量/总供应/通胀率/市值排名	CoinGecko
crypto_get_protocol_tvl	protocol_name	TVL/历史TVL/链分布/排名	DeFiLlama
crypto_get_protocol_revenue	protocol_name	协议收入/手续费/年化收入/30d收入	DeFiLlama
crypto_get_stablecoin_data	stablecoin_name或all	市值/发行量/链分布/增长趋势	DeFiLlama Stables
crypto_get_defi_yields	protocol_name, chain	DeFi收益率/APY/TVL	DeFiLlama Yields
crypto_get_l2_data	l2_name或all	Layer2 TVL/排名/费用	DeFiLlama
crypto_get_token_holders	contract_address, chain	持仓集中度/前10大地址/持仓比例	Etherscan API
crypto_get_token_unlocks	symbol	解锁日历/归属计划/团队和VC解锁量	DeFiLlama
crypto_get_derivatives	symbol	合约持仓/多空比/资金费	CCXT(Binance)

工具名称	参数	返回内容	主数据源
		率/爆仓数据	
crypto_get_dex_volume	chain或protocol	DEX交易量/排名/市占率	DeFiLlama
crypto_get_chain_comparison	chains(列表)	多链对比: TVL/用户数/交易数/费用收入	DeFiLlama
crypto_get_governance	protocol_name	DAO治理: 最近提案/投票结果/参与率	Snapshot API
crypto_get_security_audit	protocol_name	审计状态/审计公司/安全评分/历史安全事件	DeFiLlama
crypto_get_token_economics	symbol	通胀率+历史销毁量+回购记录+质押奖励率+代币分配比例(团队/VC/社区/国库)	CoinGecko+DeFiLlama+Ethersca 事件)
crypto_get_sustainability_ratio	protocol_name	30天真实手续费收入 vs 30天代币激励排放价值 → 可持续性比率(>1为可持续)+历史趋势	DeFiLlama(fees)+CoinGecko(代币 算排放)

crypto_get_token_economics说明： 服务于Web3框架第九节"经济模型行不行"。判断代币是否有价值捕获能力（回购/销毁/分红）以及激励机制是否兼容业务飞轮。内部直接调用shared/adapters/coingecko.py和defillama.py获取数据，不通过MCP协议调用同Server其他工具。

crypto_get_sustainability_ratio说明： 服务于Web3框架第八节"生态繁荣度"。判断协议的真实手续费收入是否能覆盖代币通胀补贴——这是判断DeFi协议是否有真实经济模型的核心指标。返回值>1表

示协议能自我维持，<1表示仍依赖代币补贴。内部直接调用shared/adapters/defillama.py(fees端点)和coingecko.py(代币价格)计算，与crypto_get_protocol_revenue共享同一个defillama adapter函数。

数据源API信息

数据源	API文档	认证	免费额度
DeFiLlama 主API	https://defillama.com/docs/api	无需Key	无限
DeFiLlama Stables	https://stablecoins.llama.fi	无需Key	无限
DeFiLlama Yields	https://yields.llama.fi	无需Key	无限
DeFiLlama DEX	https://api.llama.fi/overview/dexs	无需Key	无限
DeFiLlama Fees	https://api.llama.fi/overview/fees	无需Key	无限
DeFiLlama Hacks	https://api.llama.fi/hacks	无需Key	无限
CoinGecko	https://docs.coingecko.com/v3.0.1/reference/introduction	Demo Key	30次/分钟
CoinMarketCap	https://coinmarketcap.com/api/	注册Key	10,000次/月
CCXT	https://docs.ccxt.com/	pip install	取决于交易所
Etherscan	https://docs.etherscan.io/	注册Key	5次/秒
Snapshot	https://docs.snapshot.org/	无需Key(GraphQL)	无限

⚠️ CoinGecko API Plan说明

CoinGecko在2024年将API分为Demo Plan（免费）和Pro Plan（付费）。

⚠️ crypto-mcp参数命名规范（symbol vs protocol_name）

crypto-mcp中有两种参数类型，开发者必须在工具描述中向Claude解释清楚区别：

```
symbol类工具 — 传入代币符号（如"BTC"/"ETH"/"UNI"）：
  crypto_get_token_price(symbol)
  crypto_get_price_history(symbol)
  crypto_get_market_data(symbol)
  crypto_get_token_holders(contract_address, chain)
  crypto_get_token_unlocks(symbol)
  crypto_get_derivatives(symbol)
  crypto_get_token_economics(symbol)

protocol_name类工具 — 传入DeFiLlama协议名（如"uniswap"/"aave"/"gmx"）：
  crypto_get_protocol_tvl(protocol_name)
  crypto_get_protocol_revenue(protocol_name)
  crypto_get_defi_yields(protocol_name, chain)
  crypto_get_dex_volume(chain或protocol)
  crypto_get_governance(protocol_name)
  crypto_get_security_audit(protocol_name)
  crypto_get_sustainability_ratio(protocol_name)

⚠️ 同一个项目的symbol和protocol_name不同！例如：
  Uniswap: symbol="UNI", protocol_name="uniswap"
  Aave:    symbol="AAVE", protocol_name="aave"
  GMX:     symbol="GMX", protocol_name="gmx"

工具描述中必须明确说明参数类型，防止Claude传错。例如：
  ✅ "protocol_name: DeFiLlama上的协议名称（如'uniswap'），注意不是代币符号"
  ❌ "name: 项目名称"
```

本项目使用Demo Plan：

- Base URL: `https://api.coingecko.com/api/v3`
- 认证方式: Header `x-cg-demo-api-key: YOUR_KEY`
- 频率限制: 30次/分钟
- 注册地址: https://www.coingecko.com/en/api/pricing （选Demo，免费）

⚠️ 加密代币符号消歧义

很多代币共享相同的ticker符号（如多个项目可能都叫"ASTER"）。

CoinGecko使用唯一的`coin_id`（如"bitcoin"、"ethereum"），但用户输入的是symbol（如"BTC"、"ETH"）。

必须实现消歧义逻辑：

```python
async def resolve_coin_id(symbol: str) -> str:
    """
    将用户输入的symbol解析为CoinGecko的coin_id。

    1. 先查本地映射表（常见币种硬编码）
    2. 本地没有则调用CoinGecko /coins/list 搜索
    3. 如果匹配到多个结果，返回所有匹配项让Claude/用户选择
    4. 不要静默选择第一个
    """
    # 常见币种硬编码映射
    COMMON_MAPPING = {
        "BTC": "bitcoin",
        "ETH": "ethereum",
        "SOL": "solana",
        "USDT": "tether",
        "USDC": "usd-coin",
        "BNB": "binancecoin",
        # ... 扩展到Top 100
    }

    if symbol.upper() in COMMON_MAPPING:
        return COMMON_MAPPING[symbol.upper()]

    # 搜索CoinGecko
    coins = await coingecko_search(symbol)
    if len(coins) == 1:
        return coins[0]["id"]
    elif len(coins) > 1:
        # 返回所有匹配项，让Claude展示给用户选择
        matches = "\n".join([f"  - {c['id']}: {c['name']} (#{c.get('market_cap_rank'
        return f"[消歧义] 找到多个匹配'{symbol}'的代币:\n{matches}\n请指定具体的coin_id重新
    else:
        return f"[未找到] 未找到符号为'{symbol}'的加密货币。请检查拼写。"
```

7.5 Server #5: macro-mcp（宏观数据服务）

优先级: ★★★（第三批开发）

工具名称	参数	返回内容	数据源
macro_get_us_gdp	period(quarterly/annual), limit	美国GDP及增速	FRED (GDP)
macro_get_us_cpi	limit	美国CPI月度及同比	FRED (CPIAUCSL)
macro_get_us_interest_rate	limit	联邦基金利率	FRED (FEDFUNDS)
macro_get_us_unemployment	limit	美国失业率	FRED (UNRATE)
macro_get_us_treasury_yield	maturity(2Y/5Y/10Y/30Y)	美国国债收益率	FRED (DGS2/5/10/30)
macro_get_china_gdp	limit	中国GDP及增速	AKShare
macro_get_china_cpi	limit	中国CPI月度	AKShare
macro_get_china_pmi	limit	中国PMI制造业/非制造业	AKShare
macro_get_china_m2	limit	中国M2及增速	AKShare
macro_get_china_social_financing	limit	中国社融规模	AKShare
macro_get_fx_rate	from_currency, to_currency	汇率	AKShare → Alpha Vantage
macro_get_commodity_price	commodity(gold/oil/silver/copper)	商品价格	AKShare → Alpha Vantage
macro_get_vix	limit	VIX恐惧指数	FRED (VIXCLS)

FRED API注册: https://fred.stlouisfed.org/docs/api/api_key.html （免费无限额度）

7.6 Server #6: sentiment-mcp（情绪数据服务）

优先级: ★★★（第三批开发）

工具名称	参数	返回内容	数据源
sentiment_get_fear_greed_us	无	CNN恐惧贪婪指数(0-100)+子指标	CNN JSON端点
sentiment_get_fear_greed_crypto	无	加密恐惧贪婪指数	Alternative.me API
sentiment_get_news_sentiment	symbol	近期新闻情绪评分	Finnhub
sentiment_get_crypto_flows	symbol(BTC/ETH)	交易所净流入流出量	CoinGecko

注: 北向/南向资金、融资融券、做空数据已在各市场Server中提供（cn_get_northbound_flow、hk_get_southbound_flow、cn_get_margin_data、us_get_short_interest），sentiment-mcp不重复实现，避免Opus在90个工具中选择混乱。

7.7 Server #7: regulatory-mcp（监管数据服务）

优先级: ★★★（第三批开发）

工具名称	参数	返回内容	数据源
reg_get_sec_filing	symbol, filing_type, limit	SEC文件列表和链接	SEC EDGAR API
reg_get_cn_announcement	symbol, limit	A股公司公告列表	Tushare
reg_get_hk_announcement	symbol, limit	港交所公告列表	AKShare
reg_get_earnings_calendar	market(us/cn/hk), days_ahead	即将财报公司列表	Finnhub / Tushare
reg_get_ipo_calendar	market	近期IPO列表	Finnhub / AKShare
reg_get_crypto_hacks	limit	近期加密安全事件	DeFiLlama Hacks API

7.8 Server #8: analyst-mcp（分析师与估值服务）

优先级: ★★★★（第二批开发）

工具名称	参数	返回内容	数据源
analyst_get_consensus	symbol, market(us/cn/hk)	共识EPS/Revenue预测(当年+未来2年)	Finnhub / Tushare
analyst_get_price_targets	symbol	目标价(高/低/均值/中位数)+评级分布	Finnhub / FMP
analyst_get_rating_changes	symbol, limit	最近评级变动(升级/降级/新覆盖)	Finnhub
analyst_get_peer_valuation	symbol, market	可比公司估值矩阵	FMP / AKShare
analyst_get_valuation_percentile	symbol, market	当前PE/PB在历史中的百分位	自行计算
analyst_get_historical_ranking	symbol, market, metric(见下方枚举), years(5)	该指标过去N年值+同行业排名+行业中位数+是否偏离历史均值	FMP/Tushare+自行计算
analyst_get_historical_ranking_batch	symbol, market, metrics(列表,默认全部10项), years(5)	**一次性返回10个指标的完整排名表**，避免循环调用10次	FMP/Tushare+自行计算
analyst_run_dcf	symbol, growth_rate(可选), wacc(可选), terminal_growth(可选)	DCF估值结果	内置公式+FMP/Tushare数据

analyst_run_dcf参数自动估算逻辑：

当被`analyst_run_multi_valuation`内部调用时，growth_rate/wacc/terminal_growth通常不会传入，此时Server必须自动估算：

```
参数自动估算规则:
  growth_rate (未传入时):
    → 优先: 分析师共识Revenue增速 (调用analyst_get_consensus)
    → 后备: 过去3年Revenue的CAGR
    → 兜底: 行业中位数增速

  wacc (未传入时):
    → 简化公式: 无风险利率 + 5.5% + 规模溢价
    → 无风险利率 = 美国10Y国债收益率
      · 数据获取: analyst-mcp直接调用shared/adapters/fred.py (series: DGS10)
      · 不依赖macro-mcp (MCP Server间不能互相调用)
      · A股用中国10Y国债 (shared/adapters/akshare_adapter获取)
      · 港股用美国10Y国债
    → 规模溢价: 市值>500亿美元=0%, 50-500亿=1%, <50亿=2%

  terminal_growth (未传入时):
    → 美股: 2.5%  |  A股: 3.0%  |  港股: 2.5%
    → Web3: 由链上收入DCF单独处理 (见下方)

Web3链上收入DCF参数 (与传统DCF不同):
  → 收入来源: DeFiLlama年化手续费收入 (不是营业收入)
  → 折现率: 无风险利率 + 加密风险溢价 (按规模分层):
      BTC/ETH等顶级资产:          无风险 + 15%
      TVL > $1B的成熟协议:         无风险 + 25%
      TVL $100M - $1B:             无风险 + 35%
      TVL < $100M的新协议:         无风险 + 50%
  → 永续增长率: 5% (加密行业增速远高于传统行业的2-3%)
  → 预测期: 3年 (加密行业不确定性高, 不用传统的5-10年)
  → 输出: 折现后的协议"公允FDV" (非股价)

返回结果必须标注所有使用的假设值, 例如:
  "DCF估值: $17.10 [假设: 增速12.5%(共识), WACC=14.0%(10Y=4.5%+ERP5.5%+小盘2%), 永续
=2.5%]"
```

| analyst_run_multi_valuation | symbol, market, **framework(growth/mature/web3)** | 按框架运行10+种估值方法，返回交叉验证表 | 组合调用 |

⚠️ **symbol格式与market参数规范（开发者必须在工具描述中向Claude说明）：**

```
symbol格式:
  美股(market=us):   纯大写字母代码（如BTGO, AAPL, PDD）
  A股(market=cn):    6位纯数字（如000001, 600519, 300750）
  港股(market=hk):   4-5位数字（如9988, 0700, 01810）
  加密(market=crypto): 大写代码（如BTC, ETH, UNI）
  ⚠️ 不支持中文名和英文全名。用户说"拼多多"时Claude需自行转换为"PDD"。

market参数处理优先级:
  1. 如果Claude传入了market → 使用传入值
  2. 如果未传入 → Server自动检测:
     · symbol是纯数字6位 → market="cn"
     · symbol含".HK"或是4-5位纯数字 → market="hk"
     · symbol是纯大写字母 → 先查FMP(有=us), 无结果查CoinGecko(有=crypto)
  3. 自动检测不确定时 → 返回提示让Claude指定market
```

analyst_get_historical_ranking 详细说明

服务于成熟公司框架"果行不行"的10题评分——每题需要判断"该指标历史上是否处于行业第一梯队，且当前恶化是否具有周期性或一次性特征"。

metric参数必须支持以下10个值（对应成熟公司框架10题）：

```
  roic            — 投入资本收益率（第1题）
  revenue_growth  — 收入增长率（第2题）
  cash_ratio      — 净现比：经营现金流/净利润（第3题）
  cost_ratio      — 成本占比：营业成本/营业收入（第4题）
  debt_ratio      — 净资产负债率（第5题）
  owner_earnings  — 股东盈余（第6题）
  retained_value  — 利润留存创造市值（第7题）
  net_income      — 净利润（第8题）
  capex_ratio     — 资本开支与折旧开销比（第9题）
  roe             — 净资产收益率（第10题）
```

```
参数: symbol="BTGO", market="us", metric="roe", years=5
返回示例:

BTGO 净资产收益率(ROE) 5年历史 vs 行业排名

  年份     |  BTGO ROE  |  行业中位数  |  行业排名  |  行业分位
  2025     |  -4.6%     |  12.3%      |  18/22    |  18%
  2024     |  52.1%     |  11.8%      |  1/22     |  100%
  2023     |  8.7%      |  10.5%      |  8/22     |  64%
  2022     |  15.3%     |  9.2%       |  3/22     |  86%
  2021     |  22.1%     |  8.8%       |  2/22     |  91%

  历史5年平均: 18.7% (行业第一梯队 ✅)
  当前(2025)偏离: -23.3个百分点
  偏离性质: BTC国库未实现亏损导致 (一次性 ✅, 非结构性)
```

可比公司确定逻辑（开发者必读）：

analyst_get_historical_ranking需要输出"行业排名18/22"——以下是确定"哪22家公司是同行业"的规则：

```
⚠️ 规模过滤规则（三个市场+Web3统一执行）：
所有可比组都必须按规模（市值/TVL）筛选±5倍范围内的标的。
不足5家时放宽到±10倍。绝不用万亿市值公司作为百亿市值公司的可比。

market=us（美股）：
  1. 调用FMP的 /stock-peers/{symbol} 获取可比公司列表
  2. FMP返回为空时，用该公司SIC代码前3位筛选同行
  3. 从结果中按市值±5倍范围过滤，保留5-15家
  4. 返回结果中标注使用的分类标准和完整可比公司列表

market=cn（A股）：
  1. 使用Tushare的申万行业分类（三级行业）
  2. 获取同三级行业所有上市公司
  3. 按市值筛选±5倍范围内的公司作为可比组
  4. 可比组不足5家时放宽到二级行业

market=hk（港股）：
  1. 使用恒生行业分类（AKShare可获取）
  2. 或yfinance的sector/industry字段
  3. 按市值±5倍过滤
  4. 不足时参考同公司的A股对标行业

Web3可比协议:
  1. 使用DeFiLlama的category字段（如"Dexes"/"Lending"/"Bridge"/"CDP"/"Yield"）
  2. 获取同category下所有协议
  3. 按TVL±5倍范围过滤，保留5-10个（⚠️ 绝不用Uniswap $5B TVL作为$5M小协议的可比）
  4. 计算可比组的P/F、P/S、FDV/TVL中位数
  5. 返回结果中标注："可比组：同赛道TVL $XM-$YM的N个协议（排除TVL>$ZM的大协议）"
```

analyst_run_multi_valuation 详细说明

这是整个系统中最重要的工具。 必须支持`framework`参数，按不同投研框架执行不同的估值方法组合。三套框架都要求"10种以上"。

framework=growth（成长性公司，14节框架）→12种方法：

#	估值方法	隐含价值计算公式	数据来源
1	分析师共识目标价	隐含价值 = 共识目标价	analyst_get_price_targets
2	EV/Revenue可比法	隐含市值 = Revenue × 行业中位数EV/Revenue → 隐含股价 = 隐含市值/总股数	analyst_get_peer_valuation
3	EV/EBITDA可比法	隐含市值 = EBITDA × 行业中位数EV/EBITDA → 隐含股价	同上
4	PE可比法	隐含股价 = EPS × 行业中位数PE	同上
5	PB可比法	隐含股价 = BPS × 行业中位数PB	同上
6	PS可比法	隐含市值 = Revenue × 行业中位数PS → 隐含股价	同上
7	PEG法	合理PE = EPS增速% × 1.0 → 隐含股价 = 合理PE × EPS	共识EPS增速 + 当前PE
8	前瞻PE法	隐含股价 = 未来1年共识EPS × 行业平均前瞻PE	共识EPS × 行业PE
9	DCF法	隐含股价 = Σ(FCF_t/(1+WACC)^t) + 终值/(1+WACC)^n / 总股数	analyst_run_dcf
10	历史PE分位法	隐含股价 = EPS × 历史25%分位PE（保守估值）	analyst_get_valuation_percentile
11	FCF Yield法	隐含市值 = FCF / 行业中位数FCF_Yield → 隐含股价	FCF/市值 vs 行业
12	格雷厄姆数字法	隐含股价 = √(22.5 × EPS × BPS)	直接计算

framework=mature（成熟公司，8步框架）→12种方法：

#	估值方法	隐含价值计算公式	数据来源
1	股息率对比法	隐含股价 = 每股股息(DPS) / 行业中位数股息率	股息+行业数据
2	净流动资产法(NCAV)	隐含股价 = (流动资产 - 总负债) / 总股数	资产负债表
3	PE对比法	隐含股价 = EPS × min(行业中位数PE, 历史中位数PE)	行业+历史
4	PS对比法	隐含市值 = Revenue × 行业中位数PS → 隐含股价	同上逻辑
5	PB对比法	隐含股价 = BPS × 行业中位数PB	同上逻辑
6	PEG法	合理PE = 盈利增长率% × 1.0 → 隐含股价 = 合理PE × EPS	PE/增长率
7	格雷厄姆数字法	隐含股价 = √(22.5 × EPS × BPS)	直接计算
8	FCF Yield法	隐含市值 = FCF / max(行业中位数Yield, 10Y国债收益率) → 隐含股价	FCF+利率
9	历史分位法	隐含股价 = EPS × 历史25%分位PE（10年历史）	历史PE序列
10	企业价值倍数法	隐含市值 = EBITDA × 行业中位数EV/EBITDA - 净负债 → 隐含股价	行业对标
11	分析师共识目标价	隐含价值 = 共识目标价	analyst_get_price_targets
12	DCF法	同growth方法#9	analyst_run_dcf

framework=web3（Web3项目，15节框架）→12种方法：

#	估值方法	隐含FDV计算公式	数据来源
1	FDV/TVL比率法	隐含FDV = TVL × 同赛道中位数(FDV/TVL)	CoinGecko + DeFiLlama
2	P/F法	隐含FDV = 年化手续费 × 同赛道中位数P/F	DeFiLlama fees
3	P/S法	隐含FDV = 年化协议收入 × 同赛道中位数P/S	DeFiLlama revenue
4	可比协议倍数法	隐含FDV = 平均(P/F隐含, P/S隐含, FDV/TVL隐含)	综合#1-3
5	通胀调整收入法	通胀调整收入 = 协议收入 - 代币排放价值; 隐含FDV = 调整收入 × 行业P/S	见计算说明
6	活跃用户/FDV法	隐含FDV = DAU × 同赛道中位数(FDV/DAU)	DeFiLlama + CoinGecko
7	开发者活跃度/FDV法	隐含FDV = 开发者数 × 同赛道中位数(FDV/Devs)	CoinGecko开发者指标
8	TVL增长率对标法	预测TVL = 当前TVL × (1+增速)^2 → 隐含FDV = 预测TVL × 当前行业FDV/TVL	历史TVL趋势
9	解锁稀释后估值法	稀释后FDV = 当前FDV × (当前流通量+12月解锁量)/当前流通量	token_unlocks
10	链上收入DCF法	隐含FDV = Σ(Fee_t/(1+r)^t) + 终值; r=无风险+分层溢价(15%-50%按TVL规模,见DCF参数说明), 永续5%, 预测3年	手续费折现
11	可持续性比率法	隐含FDV = 真实收入 × (1/可持续性比率) × 行业P/S	sustainability_ratio
12	TAM渗透率法	隐含FDV = 赛道TAM × 参考渗透率(同赛道龙头市占率)	行业报告+DeFiLlama

Web3可比协议确定逻辑： 见上方"可比公司确定逻辑"中的"Web3可比协议"部分（按TVL±5倍过滤）。

⚠️ **方法适用性条件（不满足时标注N/A+原因，不崩溃）：**

```
需要EPS > 0:        PE可比、PEG、前瞻PE、格雷厄姆（EPS为负时标注"当前EPS为负，不适用"）
需要BPS > 0:        PB可比、格雷厄姆（累计亏损导致净资产为负时标注N/A）
需要DPS > 0:        股息率法（无分红公司标注"该公司无分红历史"）
需要FCF > 0:        FCF Yield、DCF（全负时用Revenue增长替代FCF做简化DCF）
需要EBITDA > 0:     EV/EBITDA（EBITDA为负时标注N/A）
需要流动资产>总负债: NCAV（为负时标注"净流动资产为负，不适用于该公司类型"）
需要手续费收入>0:  P/F、通胀调整收入、链上DCF、可持续性比率（新协议无收入时标N/A）
```

Web3估值方法#5"代币通胀调整收入"的计算说明：

```
"代币激励排放价值"是协议为了吸引TVL/用户而发放的代币补贴，
它等于"该时期新释放的代币数量 × 代币当前价格"。

计算方法（按数据可得性选择）：

方法A（精确，需要解锁数据）：
  月度排放量 = crypto_get_token_unlocks 返回的月度计划释放量
  月度排放价值 = 月度排放量 × 当月均价（CoinGecko）
  年化排放价值 = 月度排放价值 × 12

方法B（简化，用供应量变化推算）：
  月度排放量 = CoinGecko total_supply 的月度变化量
  年化排放价值 = 月度变化量 × 12 × 当前价格

方法C（最简，用FDV与市值差推算）：
  年化排放价值 ≈ （FDV - 市值） / 剩余归属期年数
  剩余归属期 = crypto_get_token_unlocks 的最后解锁日期 - 今天

最终公式：
  通胀调整后收入 = 年化协议收入(DeFiLlama) - 年化代币排放价值
  通胀调整收入倍数 = FDV / 通胀调整后收入
  （负值说明协议处于"补贴倒挂"，即烧钱买TVL阶段）
```

性能要求：缓存命中 < 3秒，缓存未命中 < 30秒。

⚠️ **analyst-mcp的数据源依赖分框架类型：**

```
framework=growth/mature（股票类）：
  → 需要：FMP + Finnhub + Tushare + AKShare + yfinance + FRED
  → 这些数据源已在其他Server中集成，analyst-mcp需要独立集成或通过内部HTTP调用其他Server

framework=web3（加密类）：
  → 额外需要：CoinGecko + DeFiLlama + TokenTerminal + CCXT + Etherscan
  → analyst-mcp必须同时集成传统金融和加密数据源的API连接
  → 加密数据源的API Key已在.env中配置（COINGECKO_API_KEY等），analyst-mcp直接读取使用

开发者注意：analyst-mcp是所有Server中数据源依赖最多的Server，
因为它需要跨市场聚合数据计算估值。请确保所有相关API连接在该Server中可用。
```

估值方法部分失败时的处理原则：

```
12种方法中部分失败是常态（新上市公司无5年历史、无分红股票无股息率等）。处理规则：
  1. 绝不因部分方法失败而整体报错——成功的方法正常展示，失败的标注原因
  2. 至少5种方法成功才输出中位数结论
  3. 少于5种成功时标注："⚠️ 有效方法不足，交叉验证可信度低，建议补充定性分析"
  4. framework参数未传入时，返回："[请指定框架] 请在参数中指定framework=growth或mature或
web3"
```

返回格式（以growth框架为例，12种方法含部分失败）：

```
═══════════════════════════════════════════════════════════════════
  BTGO 多维度估值交叉验证 [框架: 成长性公司]
  当前股价: $7.67 | 市值: $886M | 货币: USD
  估值日期: 2026-03-28
═══════════════════════════════════════════════════════════════════
  #  | 估值方法                | 隐含价值    | vs当前   | 状态
  1  | 分析师共识目标价         | $15.50     | +102%   | ✓
  2  | EV/Revenue可比          | $14.20     | +85%    | ✓
  3  | EV/EBITDA可比           | $16.80     | +119%   | ✓
  4  | PE可比                  | N/A        | —       | × 当前PE为负
  5  | PB可比                  | $9.80      | +28%    | ✓
  6  | PS可比                  | $12.30     | +60%    | ✓
  7  | PEG法                   | N/A        | —       | × 当前EPS为负
  8  | 前瞻PE(FY2027)          | $13.50     | +76%    | ✓
  9  | DCF                     | $17.10     | +123%   | ✓
     |   └ 假设: 增速12.5%(共识) WACC=14.0% 永续=2.5%
  10 | 历史PE 25%分位          | $11.20     | +46%    | ✓
  11 | FCF Yield               | $10.50     | +37%    | ✓
  12 | 格雷厄姆数字            | N/A        | —       | × EPS为负无法计算
  ─────────────────────────────────────────────────────────────────
  有效方法: 9/12 | 无效: 3/12 (PE/PEG/格雷厄姆因EPS为负不适用)
  中位数隐含价值: $13.50 (+76%) [基于9种有效方法]
  均值隐含价值: $13.43 (+75%)
  最低估值: $9.80 (PB可比) | 最高估值: $17.10 (DCF)
  9种方法均高于当前价格: ✅ 是

  数据来源: FMP+Finnhub+FRED | 缓存状态: 部分命中(2026-03-28 10:00 UTC)
═══════════════════════════════════════════════════════════════════
```

Web3框架(framework="web3")返回格式示例:

```
═══════════════════════════════════════════════════════════════
ASTER 多维度估值交叉验证 [框架: Web3项目]
当前币价: $0.15 | 市值: $45M | FDV: $150M | TVL: $320M
估值日期: 2026-03-28
═══════════════════════════════════════════════════════════════
#  | 估值方法                | 隐含FDV     | vs当前FDV | 状态
1  | FDV/TVL (vs同类)        | $224M       | +49%      | ✓
2  | P/F (FDV/年化手续费)    | $180M       | +20%      | ✓
3  | P/S (FDV/年化协议收入)  | $195M       | +30%      | ✓
4  | 可比协议倍数            | $210M       | +40%      | ✓
5  | 通胀调整收入倍数        | $165M       | +10%      | ✓
6  | 活跃用户/FDV            | N/A         | —         | × 用户数据不可得
7  | 开发者活跃度/FDV        | $200M       | +33%      | ✓
8  | TVL增长率对标           | $250M       | +67%      | ✓
9  | 解锁稀释后FDV           | $130M       | -13%      | ⚠️ 稀释后低于当前
10 | 链上收入DCF             | $175M       | +17%      | ✓
11 | 可持续性比率定价        | $155M       | +3%       | ✓
12 | TAM渗透率反推           | $300M       | +100%     | ✓
──────────────────────────────────────────────────────────────
有效方法: 11/12 | 无效: 1/12
中位数隐含FDV: $195M (+30%) [基于11种有效方法]
当前FDV $150M 低于中位数估值: ✅ 是 (折价23%)
⚠️ 注意: 方法#9显示未来12个月解锁将稀释FDV至$130M

可持续性比率: 0.73 (手续费收入尚未覆盖代币激励, 趋势改善中)
数据来源: DeFiLlama+CoinGecko+TokenTerminal
═══════════════════════════════════════════════════════════════
```

注意：Web3框架的返回格式使用**FDV**而非股价，**TVL**而非市值，**P/F**而非PE。开发者必须按框架类型切换输出格式。

成熟公司框架(framework="mature")返回格式示例：

```
═══════════════════════════════════════════════════════════════
  PDD 多维度估值交叉验证 [框架: 成熟公司]
  当前股价: $102.30 | 市值: $143B | 货币: USD
  估值日期: 2026-03-28
═══════════════════════════════════════════════════════════════
  #  | 估值方法                | 隐含价值    | vs当前   | 状态
  1  | 股息率对比法            | N/A         | —        | × 该公司无分红历史
  2  | 净流动资产法(NCAV)      | $28.50      | -72%     | ⚠️ 远低于股价（非资产型公司）
  3  | PE对比法                | $125.80     | +23%     | ✓
  4  | PS对比法                | $118.60     | +16%     | ✓
  5  | PB对比法                | $95.20      | -7%      | ✓
  6  | PEG法                   | $135.40     | +32%     | ✓
  7  | 格雷厄姆数字法          | $88.90      | -13%     | ✓
  8  | FCF Yield法             | $142.10     | +39%     | ✓
  9  | 历史PE分位法(10年)      | $110.50     | +8%      | ✓
  10 | 企业价值倍数法(EV/EBITDA) | $130.20 | +27%     | ✓
  11 | 分析师共识目标价        | $148.00     | +45%     | ✓
  12 | DCF法                   | $155.30     | +52%     | ✓
     |  └ 假设: 增速15%(共识) WACC=11.5% 永续=2.5%
  ────────────────────────────────────────────────────────────
  有效方法: 10/12 | 无效: 2/12 (股息率/NCAV不适用于平台型公司)
  中位数隐含价值: $128.00 (+25%) [基于10种有效方法]
  均值隐含价值: $125.00 (+22%)
  最低估值: $88.90 (格雷厄姆) | 最高估值: $155.30 (DCF)
  10种方法中8种高于当前价格: ✅ 偏低估

  数据来源: FMP+Finnhub+FRED | 缓存状态: 全部命中
═══════════════════════════════════════════════════════════════
```

---

8. 核心设计要求

8.1 数据源故障自动切换

每个工具内部必须实现"主数据源 → 后备数据源"的自动切换逻辑:

```python
```

```python
import asyncio
import logging

logger = logging.getLogger(__name__)

async def get_data_with_fallback(symbol: str, sources: list):
    """
    按顺序尝试多个数据源，第一个成功就返回。

    sources格式: [("FMP", fmp_func), ("yfinance", yf_func), ("AlphaVantage", av_func)
    """
    errors = []
    for source_name, source_func in sources:
        # 先检查API额度
        if not await check_quota(source_name):
            logger.warning(f"⚠ {source_name} 额度已用完，跳过")
            continue

        try:
            data = await source_func(symbol)
            if data is not None and (isinstance(data, str) and len(data) > 0 or isin
                logger.info(f"✓ {source_name} 成功获取 {symbol}")
                await increment_quota(source_name)
                return data
            else:
                logger.warning(f"× {source_name} 返回空数据 {symbol}")
        except Exception as e:
            logger.warning(f"× {source_name} 获取 {symbol} 失败: {type(e).__name__}:
            errors.append(f"{source_name}: {e}")
            continue

    # 全部失败
    error_details = "; ".join(errors)
    return (
        f"[数据获取失败] 无法获取 {symbol} 的数据\n"
        f"尝试过的数据源: {', '.join([s[0] for s in sources])}\n"
        f"错误详情: {error_details}\n"
        f"建议: Claude可使用web_search搜索相关数据补充"
    )
```

8.2 Redis缓存策略

⚠️ 所有工具必须支持`force_refresh`可选参数（默认False）。危机投资场景中标的可能刚暴跌，委

托人需要最新数据而非缓存数据。

```python
# 所有工具的参数签名应包含force_refresh:
@mcp.tool()
async def us_get_income_statement(symbol: str, period: str = "annual",
                                  limit: int = 3, force_refresh: bool = False) -> s
    """...(description)...
    force_refresh: 设为True跳过缓存直接获取最新数据（危机分析时使用）"""
```

```python
```

```python
import hashlib
import json

CACHE_TTL = {
    "stock_price": 60,              # 1分钟
    "price_history": 86400,          # 24小时
    "financial_statements": 604800,  # 7天
    "financial_ratios": 86400,       # 24小时
    "analyst_ratings": 21600,        # 6小时
    "insider_trades": 86400,         # 24小时
    "shareholders": 86400,           # 24小时
    "macro_data": 43200,             # 12小时
    "crypto_price": 60,              # 1分钟
    "crypto_tvl": 3600,              # 1小时
    "news": 1800,                    # 30分钟
    "multi_valuation": 21600,        # 6小时
}

def cache_key(source: str, endpoint: str, symbol: str, **params) -> str:
    """
    缓存Key基于数据源+端点，而非Server名称。
    这确保跨Server缓存共享生效：
    us-stocks-mcp 和 analyst-mcp 调用 fmp.get_income_statement("BTGO")
    生成的Key完全相同，后者直接读缓存。

    示例Key: "fmp:income_statement:BTGO:annual_3_a1b2c3d4"
    """
    params_hash = hashlib.md5(json.dumps(params, sort_keys=True).encode()).hexdigest
    return f"{source}:{endpoint}:{symbol}:{params_hash}"

async def get_cached_or_fetch(redis, key: str, ttl: int, fetch_func, force_refresh:
    """先查缓存，未命中则调用fetch_func并缓存结果"""
    # ⚠️ force_refresh=True时跳过缓存（危机分析场景）
    if not force_refresh:
        # ⚠️ Redis容错：Redis不可用时继续工作（无缓存模式）
        try:
            cached = await redis.get(key)
            if cached:
                logger.debug(f"缓存命中: {key}")
                return cached.decode()
        except Exception as e:
            logger.warning(f"Redis不可用，跳过缓存: {e}")
    else:
```

```
        logger.info(f"强制刷新: {key}")

    result = await fetch_func()

    # 尝试写入缓存，失败也不影响返回结果
    try:
        if result and not result.startswith("[数据获取失败]"):
            await redis.setex(key, ttl, result)
    except Exception as e:
        logger.warning(f"Redis写入失败，结果仍然返回: {e}")

    return result
```

8.3 API额度管理

```python
DAILY_LIMITS = {
    "fmp": 250,
    "finnhub": 86400,       # 60/min ≈ 86400/day
    "alpha_vantage": 25,
    "coingecko": 43200,     # 30/min
    "marketaux": 100,
    "coinmarketcap": 333,   # 10000/month
    "etherscan": 432000,    # 5/sec
}

async def check_quota(source_name: str) -> bool:
    key = f"quota:{source_name}:{today_str()}"
    current = await redis.get(key)
    if current is None:
        return True
    return int(current) < DAILY_LIMITS.get(source_name, 999999)

async def increment_quota(source_name: str):
    key = f"quota:{source_name}:{today_str()}"
    await redis.incr(key)
    await redis.expire(key, 86400)
```

8.4 返回数据格式

所有工具返回给Claude的数据必须是**人类可读的文本格式**（不是原始JSON）。

正确的返回格式:

```
BitGo Holdings (BTGO) FY2025 利润表 [货币: USD]

                          FY2025          FY2024          YoY变化
总收入                    $16,150M        $3,080M         +424.0%
营业成本                  $15,951M        $2,836M         +462.6%
毛利润                    $199.1M         $244.0M         -18.4%
营业费用                  $184.2M         $158.0M         +16.6%
营业利润                  $14.9M          $86.0M          -82.7%
净利润                    ($14.8M)        $156.5M         反转
EPS (基本)                ($0.13)         $1.35           反转

数据来源: FMP | 获取时间: 2026-03-28 14:30 UTC | 实时获取
```

数据来源标注规范（开发者必须遵守）：

```
每个工具返回的末尾必须标注数据来源和真实获取时间。区分四种状态:

缓存命中:    "数据来源: FMP | 获取时间: 2026-03-25 10:00 UTC | 缓存(剩余4天)"
实时获取:    "数据来源: FMP | 获取时间: 2026-03-28 14:30 UTC | 实时获取"
后备源:      "数据来源: yfinance(后备) | 获取时间: 2026-03-28 14:30 UTC | ⚠️ FMP不可用"
强制刷新:    "数据来源: FMP | 获取时间: 2026-03-28 14:30 UTC | 强制刷新"

⚠️ "获取时间"必须是数据实际从API获取的时间，不是当前时间。
   缓存数据的获取时间是写入缓存的时间，不是读取缓存的时间。
   这在危机投资场景中至关重要——委托人需要知道数据的真实新鲜度。
```

货币和时区规范:

```
货币显示:
  - 美股: $ (USD)
  - A股: ¥ (CNY)
  - 港股: HK$ (HKD)
  - 加密: $ (USD)
  - 每个返回结果必须标注货币单位

时间显示:
  - 日期格式: YYYY-MM-DD
  - 美股时间: 标注 EST/EDT
  - A股时间: 标注 CST (UTC+8)
  - 加密: 标注 UTC
```

8.5 数据验证

API返回的原始数据可能包含异常值。所有数据在格式化输出前必须经过验证:

⚠️ **币种对齐规则（估值计算前必须执行，否则PE/PB等指标会严重错误）**

PDD股价是USD但财务报表以RMB记账，9988.HK股价是HKD但报表以USD记账。 如果不做汇率转换直接算PE=股价/EPS，PDD的PE会算成2.8而非实际的20.1——差7倍。

```python
async def align_currency(symbol: str, market: str, price_data: dict, financial_data:
    """
    确保股价和财务数据使用同一币种后再计算估值指标。

    处理方式:
      1. 获取报表币种（FMP: reportedCurrency / yfinance: financialCurrency）
      2. 获取交易币种（FMP: currency / yfinance: currency）
      3. 不一致时 → 调用shared/adapters/fred.py或AKShare获取汇率
      4. 将财务数据统一转换为交易币种
      5. 返回结果中标注使用的汇率

    常见不一致案例:
      PDD:     交易USD，报表RMB → EPS÷USD/CNY汇率
      9988.HK: 交易HKD，报表USD → EPS×USD/HKD汇率
      0700.HK: 交易HKD，报表RMB → EPS÷CNY/HKD汇率
      BTGO:    交易USD，报表USD → 无需转换
    """
```

此函数必须在`analyst_run_multi_valuation`的**每种估值方法计算前**调用。 返回的估值表格中标注: "⚠️ 报表币种(RMB)已按汇率7.25转换为交易币种(USD)"

```python
```

```python
def validate_financial_data(data: dict) -> dict:
    """清洗和验证财务数据"""
    cleaned = {}
    for key, value in data.items():
        # 处理NaN/None
        if value is None or (isinstance(value, float) and math.isnan(value)):
            cleaned[key] = "N/A"
            continue

        # 数值合理性检查
        if key == "market_cap" and isinstance(value, (int, float)) and value < 0:
            cleaned[key] = "N/A (异常值)"
            continue

        if key in ("pe_ratio", "pb_ratio") and isinstance(value, (int, float)):
            if abs(value) > 10000:
                cleaned[key] = f"{value:.1f} (⚠️ 异常高)"
                continue

        cleaned[key] = value

    return cleaned
```

8.6 错误处理

```
[数据获取失败] 无法获取 BTGO 的分析师评级数据
原因: FMP返回空结果, Finnhub返回HTTP 429
建议: Claude可使用web_search搜索 "BTGO analyst rating target price" 补充
```

8.7 日志记录

使用Python的`RotatingFileHandler`防止日志文件膨胀:

```python
```

```python
import logging
from logging.handlers import RotatingFileHandler

def setup_logging(server_name: str):
    logger = logging.getLogger(server_name)
    logger.setLevel(logging.INFO)

    # 控制台输出
    console = logging.StreamHandler()
    console.setFormatter(logging.Formatter(
        "%(asctime)s [%(levelname)s] %(name)s | %(message)s"
    ))
    logger.addHandler(console)

    # 文件输出（单文件10MB上限，保留5个历史文件）
    file_handler = RotatingFileHandler(
        f"/app/logs/{server_name}.log",
        maxBytes=10*1024*1024,  # 10MB
        backupCount=5
    )
    file_handler.setFormatter(logging.Formatter(
        "%(asctime)s [%(levelname)s] %(name)s | %(message)s"
    ))
    logger.addHandler(file_handler)

    return logger
```

日志格式:

```
2026-03-28 10:15:23 [INFO] us-stocks | us_get_income_statement | BTGO | FMP | 成功
| 1.2s | 缓存未命中
2026-03-28 10:15:24 [INFO] us-stocks | us_get_analyst_ratings | BTGO | Finnhub |
成功 | 0.8s | 缓存命中
2026-03-28 10:15:25 [WARN] us-stocks | us_get_insider_trades | BTGO | FMP | 失败
(429) | 切换Finnhub
```

8.8 标的存在性预检

用户可能输入不存在的股票代码（如"XYZABC"）。如果不做预检，15-20个工具会依次返回各自格式的错误信息，Claude输出一份乱七八糟的"报告"。

要求： 在shared/validators.py中实现统一的标的验证函数，每个工具的第一步调用它：

```python
# shared/validators.py
async def validate_symbol(symbol: str, market: str) -> tuple[bool, str]:
    """
    验证标的是否存在。返回 (是否存在, 错误信息)。
    每个工具在执行业务逻辑前先调用此函数。
    """
    if market == "us":
        # 用FMP或Finnhub查company profile
        profile = await fmp.get_profile(symbol)
        if not profile:
            return False, f"[标的不存在] 未找到美股代码'{symbol}'。请检查是否拼写正确。"
    elif market == "cn":
        # 用AKShare查A股列表
        exists = await akshare_adapter.check_stock_exists(symbol)
        if not exists:
            return False, f"[标的不存在] 未找到A股代码'{symbol}'。A股代码为6位数字（如60051
    elif market == "hk":
        # 用yfinance检查 symbol.HK 是否存在
        profile = await yahoo.get_profile(f"{symbol}.HK")
        if not profile:
            return False, f"[标的不存在] 未找到港股代码'{symbol}'。港股代码为4-5位数字（如99
    elif market == "crypto":
        # 用CoinGecko resolve
        coin_id = await coingecko.resolve_coin_id(symbol)
        if coin_id.startswith("[未找到]"):
            return False, coin_id
    return True, ""
```

这样当标的不存在时，Claude收到的是一条统一的错误信息，可以直接告知用户，而不是15条不同格式的错误。

9.0 首次部署（从零到运行，按此顺序执行）

```bash
```

```
# 1. 购买服务器（推荐阿里云香港ECS 4C8G，见9.6节）

# 2. SSH登录服务器，安装Docker
curl -fsSL https://get.docker.com | sh
apt install docker-compose-plugin

# 3. 克隆代码仓库
git clone git@github.com:your-repo/mcp-investment-research.git
cd mcp-investment-research

# 4. 配置环境变量
cp .env.example .env
vim .env    # 填入所有API Key和密码

# 5. 获取SSL证书
apt install certbot
certbot certonly --standalone -d your-domain.com
mkdir -p nginx/ssl
cp /etc/letsencrypt/live/your-domain.com/fullchain.pem nginx/ssl/cert.pem
cp /etc/letsencrypt/live/your-domain.com/privkey.pem nginx/ssl/key.pem

# 6. 启动全部服务
docker compose up -d --build

# 7. 验证
docker compose ps                          # 确认全部容器running
docker compose logs --tail=20              # 检查启动日志无报错
curl -k https://localhost/us-stocks/mcp    # 应返回405（POST-only，说明Server在运行）

# 8. 配置Claude连接（见第6节）
#    方式A（claude.ai Research模式 — 主要使用方式）:
#      打开claude.ai → Settings → Integrations
#      依次添加8个MCP Server连接（每个单独添加）:
#        名称: us-stocks    URL: https://your-domain.com/us-stocks/mcp
#        名称: cn-stocks    URL: https://your-domain.com/cn-stocks/mcp
#        名称: hk-stocks    URL: https://your-domain.com/hk-stocks/mcp
#        名称: crypto       URL: https://your-domain.com/crypto/mcp
#        名称: macro        URL: https://your-domain.com/macro/mcp
#        名称: sentiment    URL: https://your-domain.com/sentiment/mcp
#        名称: regulatory   URL: https://your-domain.com/regulatory/mcp
#        名称: analyst      URL: https://your-domain.com/analyst/mcp
#      每个都设置Bearer Token = .env中的MCP_AUTH_TOKEN
#
```

```
#    方式B (Claude Desktop — 开发调试):
#      编辑claude_desktop_config.json (见第6.2节)
#
# 9. 在Claude中测试: 选择Research功能 + Opus 4.6模型
#    输入: "用us_get_stock_price查询AAPL的股价"

# 10. 设置SSL自动续期 (注意用追加方式, 不覆盖现有cron任务)
(crontab -l 2>/dev/null; echo "0 3 1,15 * * certbot renew --quiet && docker compose
```

9.1 Dockerfile（每个Server各一份）

每个Server有自己的Dockerfile（内容几乎相同，只有COPY行和端口不同）。

us-stocks-mcp/Dockerfile 示例：

```dockerfile
FROM python:3.11-slim
WORKDIR /app
ENV PYTHONPATH=/app

RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*

COPY us-stocks-mcp/requirements.txt ./requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

COPY shared/ /app/shared/
COPY us-stocks-mcp/*.py /app/

RUN mkdir -p /app/logs
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
    CMD python -c "import socket; s=socket.socket(); s.settimeout(2); s.connect(('lo
CMD python server.py --transport streamable-http --port 9001
```

其他Server照此格式，替换三处：`us-stocks-mcp` → 对应目录名，`9001` → 对应端口号。

```
各Server的Dockerfile差异点:
  us-stocks-mcp:     COPY us-stocks-mcp/*.py    端口9001
  cn-stocks-mcp:     COPY cn-stocks-mcp/*.py    端口9002
  hk-stocks-mcp:     COPY hk-stocks-mcp/*.py    端口9003
  crypto-mcp:        COPY crypto-mcp/*.py       端口9004
  macro-mcp:         COPY macro-mcp/*.py        端口9005
  sentiment-mcp:     COPY sentiment-mcp/*.py    端口9006
  regulatory-mcp:    COPY regulatory-mcp/*.py   端口9007
  analyst-mcp:       COPY analyst-mcp/*.py      端口9008
```

9.2 Docker Compose配置

```yaml
```

```yaml
# docker-compose.yml
# 注: Docker Compose V2不需要version字段

services:
  redis:
    image: redis:7-alpine
    command: redis-server --requirepass ${REDIS_PASSWORD:-defaultpass123} --maxmemor
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
    environment:
      - REDIS_PASSWORD=${REDIS_PASSWORD:-defaultpass123}
    healthcheck:
      test: ["CMD-SHELL", "redis-cli -a ${REDIS_PASSWORD:-defaultpass123} ping | gre
      interval: 10s
      timeout: 5s
      retries: 3
    deploy:
      resources:
        limits:
          memory: 256M
    restart: always

  us-stocks-mcp:
    build:
      context: .
      dockerfile: us-stocks-mcp/Dockerfile
    ports:
      - "9001:9001"
    environment:
      - PORT=9001
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
    env_file: .env
    depends_on:
      redis:
        condition: service_healthy
    volumes:
      - ./logs/us-stocks:/app/logs
      - ./shared:/app/shared
    restart: always

  cn-stocks-mcp:
```

```yaml
    build:
      context: .
      dockerfile: cn-stocks-mcp/Dockerfile
    ports:
      - "9002:9002"
    environment:
      - PORT=9002
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
    env_file: .env
    depends_on:
      redis:
        condition: service_healthy
    volumes:
      - ./logs/cn-stocks:/app/logs
      - ./shared:/app/shared
    restart: always

  hk-stocks-mcp:
    build:
      context: .
      dockerfile: hk-stocks-mcp/Dockerfile
    ports:
      - "9003:9003"
    environment:
      - PORT=9003
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
    env_file: .env
    depends_on:
      redis:
        condition: service_healthy
    volumes:
      - ./logs/hk-stocks:/app/logs
      - ./shared:/app/shared
    restart: always

  crypto-mcp:
    build:
      context: .
      dockerfile: crypto-mcp/Dockerfile
    ports:
      - "9004:9004"
    environment:
      - PORT=9004
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
```

```yaml
    env_file: .env
    depends_on:
      redis:
        condition: service_healthy
    volumes:
      - ./logs/crypto:/app/logs
      - ./shared:/app/shared
    restart: always

  macro-mcp:
    build:
      context: .
      dockerfile: macro-mcp/Dockerfile
    ports:
      - "9005:9005"
    environment:
      - PORT=9005
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
    env_file: .env
    depends_on:
      redis:
        condition: service_healthy
    volumes:
      - ./logs/macro:/app/logs
      - ./shared:/app/shared
    restart: always

  sentiment-mcp:
    build:
      context: .
      dockerfile: sentiment-mcp/Dockerfile
    ports:
      - "9006:9006"
    environment:
      - PORT=9006
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
    env_file: .env
    depends_on:
      redis:
        condition: service_healthy
    volumes:
      - ./logs/sentiment:/app/logs
      - ./shared:/app/shared
    restart: always
```

```yaml
  regulatory-mcp:
    build:
      context: .
      dockerfile: regulatory-mcp/Dockerfile
    ports:
      - "9007:9007"
    environment:
      - PORT=9007
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
    env_file: .env
    depends_on:
      redis:
        condition: service_healthy
    volumes:
      - ./logs/regulatory:/app/logs
      - ./shared:/app/shared
    restart: always

  analyst-mcp:
    build:
      context: .
      dockerfile: analyst-mcp/Dockerfile
    ports:
      - "9008:9008"
    environment:
      - PORT=9008
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
    env_file: .env
    depends_on:
      redis:
        condition: service_healthy
    volumes:
      - ./logs/analyst:/app/logs
      - ./shared:/app/shared
    restart: always

  nginx:
    image: nginx:alpine
    ports:
      - "443:443"
      - "80:80"
    volumes:
      - ./nginx/nginx.conf.template:/etc/nginx/templates/default.conf.template
```

```yaml
      - ./nginx/ssl:/etc/nginx/ssl
    environment:
      - MCP_AUTH_TOKEN=${MCP_AUTH_TOKEN}
      # 限制envsubst只替换MCP_AUTH_TOKEN，不吞掉nginx内建变量（$host/$remote_addr等）
      - NGINX_ENVSUBST_FILTER=MCP_AUTH_TOKEN
    depends_on:
      - us-stocks-mcp
      - cn-stocks-mcp
      - hk-stocks-mcp
      - crypto-mcp
      - macro-mcp
      - sentiment-mcp
      - regulatory-mcp
      - analyst-mcp
    restart: always

volumes:
  redis_data:
```

9.3 环境变量文件

```bash
```

```
# .env 文件（不要提交到Git）

# === 安全认证 ===
MCP_AUTH_TOKEN=your_secret_bearer_token_here
REDIS_PASSWORD=your_redis_password_here

# === 美股数据源 ===
FMP_API_KEY=
FINNHUB_API_KEY=
ALPHA_VANTAGE_API_KEY=
MARKETAUX_API_KEY=
SEC_EDGAR_USER_AGENT=YourName your@email.com

# === A股数据源 ===
TUSHARE_TOKEN=

# === 加密数据源 ===
COINGECKO_API_KEY=
COINMARKETCAP_API_KEY=
ETHERSCAN_API_KEY=

# === 宏观数据源 ===
FRED_API_KEY=
```

9.4 Nginx配置（含SSE支持和认证）

⚠️ **Nginx不支持环境变量。** 本项目使用nginx:alpine镜像内置的envsubst功能自动替换模板中的变量。

docker-compose中nginx的配置已在9.2节定义（使用`/etc/nginx/templates/`目录的模板自动处理机制）。以下是需要保存为`nginx/nginx.conf.template`的模板文件内容：

⚠️ **envsubst变量冲突防护：** docker-compose中已设置`NGINX_ENVSUBST_FILTER=MCP_AUTH_TOKEN`，确保只有`${MCP_AUTH_TOKEN}`会被替换，nginx自身的`$host`、`$remote_addr`等变量不会被吞掉。

```
nginx
```

```
# nginx/nginx.conf.template
# 速率限制（防止异常调用耗尽API额度）
limit_req_zone $binary_remote_addr zone=mcp_limit:10m rate=10r/s;

# HTTP → HTTPS重定向（防止80端口泄露服务器信息）
server {
    listen 80;
    server_name your-domain.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name your-domain.com;

    ssl_certificate     /etc/nginx/ssl/cert.pem;
    ssl_certificate_key /etc/nginx/ssl/key.pem;

    # === MCP Server路由 ===
    # 每个location包含：Bearer Token认证 + SSE支持 + 速率限制

    location /us-stocks/mcp {
        # Bearer Token验证
        if ($http_authorization != "Bearer ${MCP_AUTH_TOKEN}") {
            return 401;
        }

        # 速率限制
        limit_req zone=mcp_limit burst=20 nodelay;

        proxy_pass http://us-stocks-mcp:9001/mcp;

        # SSE/Streaming 必需配置
        proxy_buffering off;
        proxy_cache off;
        proxy_set_header Connection '';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_http_version 1.1;
        chunked_transfer_encoding off;
        proxy_read_timeout 300s;
        proxy_send_timeout 300s;
    }
```

```
    location /cn-stocks/mcp {
        if ($http_authorization != "Bearer ${MCP_AUTH_TOKEN}") { return 401; }
        limit_req zone=mcp_limit burst=20 nodelay;
        proxy_pass http://cn-stocks-mcp:9002/mcp;
        proxy_buffering off;
        proxy_cache off;
        proxy_set_header Connection '';
        proxy_http_version 1.1;
        chunked_transfer_encoding off;
        proxy_read_timeout 300s;
    }

    location /hk-stocks/mcp {
        if ($http_authorization != "Bearer ${MCP_AUTH_TOKEN}") { return 401; }
        limit_req zone=mcp_limit burst=20 nodelay;
        proxy_pass http://hk-stocks-mcp:9003/mcp;
        proxy_buffering off;
        proxy_cache off;
        proxy_set_header Connection '';
        proxy_http_version 1.1;
        chunked_transfer_encoding off;
        proxy_read_timeout 300s;
    }

    location /crypto/mcp {
        if ($http_authorization != "Bearer ${MCP_AUTH_TOKEN}") { return 401; }
        limit_req zone=mcp_limit burst=20 nodelay;
        proxy_pass http://crypto-mcp:9004/mcp;
        proxy_buffering off;
        proxy_cache off;
        proxy_set_header Connection '';
        proxy_http_version 1.1;
        chunked_transfer_encoding off;
        proxy_read_timeout 300s;
    }

    location /macro/mcp {
        if ($http_authorization != "Bearer ${MCP_AUTH_TOKEN}") { return 401; }
        limit_req zone=mcp_limit burst=20 nodelay;
        proxy_pass http://macro-mcp:9005/mcp;
        proxy_buffering off;
        proxy_cache off;
        proxy_set_header Connection '';
```

```
        proxy_http_version 1.1;
        chunked_transfer_encoding off;
        proxy_read_timeout 300s;
    }

    location /sentiment/mcp {
        if ($http_authorization != "Bearer ${MCP_AUTH_TOKEN}") { return 401; }
        limit_req zone=mcp_limit burst=20 nodelay;
        proxy_pass http://sentiment-mcp:9006/mcp;
        proxy_buffering off;
        proxy_cache off;
        proxy_set_header Connection '';
        proxy_http_version 1.1;
        chunked_transfer_encoding off;
        proxy_read_timeout 300s;
    }

    location /regulatory/mcp {
        if ($http_authorization != "Bearer ${MCP_AUTH_TOKEN}") { return 401; }
        limit_req zone=mcp_limit burst=20 nodelay;
        proxy_pass http://regulatory-mcp:9007/mcp;
        proxy_buffering off;
        proxy_cache off;
        proxy_set_header Connection '';
        proxy_http_version 1.1;
        chunked_transfer_encoding off;
        proxy_read_timeout 300s;
    }

    location /analyst/mcp {
        if ($http_authorization != "Bearer ${MCP_AUTH_TOKEN}") { return 401; }
        limit_req zone=mcp_limit burst=20 nodelay;
        proxy_pass http://analyst-mcp:9008/mcp;
        proxy_buffering off;
        proxy_cache off;
        proxy_set_header Connection '';
        proxy_http_version 1.1;
        chunked_transfer_encoding off;
        proxy_read_timeout 300s;
    }
}
```

9.5 SSL证书获取

```
SSL证书获取方式（二选一）：

方案A: Let's Encrypt免费证书（生产环境推荐）
  1. 安装certbot: apt install certbot
  2. 获取证书: certbot certonly --standalone -d your-domain.com
  3. 证书路径: /etc/letsencrypt/live/your-domain.com/fullchain.pem + privkey.pem
  4. 自动续期: certbot renew --dry-run 确认后添加cron任务
     0 3 * * * certbot renew --quiet && docker restart nginx

方案B: 自签名证书（开发阶段临时用）
  openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
      -keyout nginx/ssl/key.pem -out nginx/ssl/cert.pem
  ⚠️ 自签名证书可能导致Claude Desktop连接时TLS验证失败
  ⚠️ 仅用于开发测试，生产环境必须用Let's Encrypt
```

9.6 服务器配置

配置项	推荐值	备注
云服务商	阿里云ECS 或 AWS EC2	国内用阿里云访问A股数据更快
实例规格	**4核8G内存**	8个Python进程+Redis+Nginx需要充足内存
系统盘	60G SSD	Docker镜像+日志+缓存
带宽	5Mbps	数据传输量不大
操作系统	Ubuntu 22.04 LTS	Docker兼容性最好
月费估计	¥400-800/月	阿里云ECS或轻量应用服务器

注: 阶段一只运行3个Server时，2核4G约¥200-400/月也够用。
全部8个Server上线后建议升级到4核8G。

⚠️ 服务器选址建议（影响API连通性）

DeFiLlama、CoinGecko、Etherscan、FMP、Finnhub等都是境外服务。服务器在中国大陆可能无法直接访问或速度极慢。

```
按主要分析市场选择服务器位置:

同时分析美股+A股+加密（推荐）:
  → 阿里云（香港）或 AWS（新加坡/东京）
  → 中国大陆和境外API都能正常访问
  → 阿里云香港轻量约¥30-50/月，ECS约¥300-500/月

主要分析A股:
  → 阿里云（中国大陆）
  → 但crypto-mcp和us-stocks-mcp需要通过代理访问境外API

主要分析美股+加密:
  → AWS或DigitalOcean（美西/美东）
  → 访问FMP/Finnhub/DeFiLlama延迟最低
```

SSL证书过期监控

Let's Encrypt证书90天有效。如果cron续期任务静默失败，证书过期后Claude Desktop会TLS握手失败、无法连接。

```
过期监控方式（推荐至少选一种）:

方案A: cron邮件告警
  0 9 * * 1 certbot certificates 2>&1 | grep -A2 "VALID\|INVALID\|days" | mail -s
"SSL证书状态" your@email.com

方案B: 免费在线监控（最简单）
  注册 UptimeRobot.com（免费50个监控）
  添加HTTPS监控: https://your-domain.com
  当站点不可达时自动发邮件/微信通知
```

9.7 日志排查

8个Server的日志分散在8个目录。以下是快速排查问题的方法:

```bash
```

```
# 方法A: 实时查看所有Server的交织日志
docker-compose logs --tail=50 --follow

# 方法B: 按Server筛选
docker-compose logs us-stocks-mcp --tail=100
docker-compose logs analyst-mcp --tail=100

# 方法C: 按关键词搜索全部日志文件
grep -r "BTGO" logs/ --include="*.log" | sort | tail -50
grep -r "失败\|ERROR\|WARN" logs/ --include="*.log" | tail -20

# 方法D: 按时间范围筛选
docker-compose logs --since="2026-03-28T10:00:00" --until="2026-03-28T10:30:00"
```

9.8 监控与告警

本系统为个人使用，不需要Prometheus+Grafana。使用基于Redis的轻量统计方案：

运行统计（在shared/stats.py中实现）：

```python
# shared/stats.py
async def record_api_call(redis, source: str, success: bool, elapsed: float):
    """记录每次API调用的统计信息"""
    date_key = f"stats:{datetime.now().strftime('%Y-%m-%d')}"
    await redis.hincrby(date_key, f"{source}:calls", 1)
    if not success:
        await redis.hincrby(date_key, f"{source}:failures", 1)
    await redis.expire(date_key, 86400 * 7)  # 保留7天统计

async def record_cache_hit(redis, hit: bool):
    date_key = f"stats:{datetime.now().strftime('%Y-%m-%d')}"
    await redis.hincrby(date_key, "cache:hits" if hit else "cache:misses", 1)
```

查看统计（在analyst-mcp中添加一个工具）：

```
analyst_get_system_stats（无参数）：
  返回：今日各数据源调用次数/失败率、缓存命中率、活跃告警
  示例输出：
    MCP系统运行状态 [2026-03-28]
    ────────────────────────────
    FMP:         45/250次 (18%) | 失败: 0 | ✅
    Finnhub:     120次 | 失败: 2 (1.7%) | ✅
    CoinGecko:   35次 | 失败: 0 | ✅
    yfinance:    8次 | 失败: 3 (37%) | ⚠️ 高失败率
    缓存命中率: 73% (267/365)
    ────────────────────────────
    ⚠️ 告警: yfinance过去1小时连续失败5次，已自动切换到FMP后备
```

故障告警规则（在shared/fallback.py中实现）：

```python
# shared/fallback.py 中增加告警逻辑
async def check_and_alert(redis, source: str):
    """检查数据源是否频繁失败，触发告警"""
    hour_key = f"alert:{source}:{datetime.now().strftime('%Y-%m-%d-%H')}"
    failures = await redis.get(hour_key)
    if failures and int(failures) >= 10:
        alert_key = f"alert:active:{source}"
        await redis.setex(alert_key, 86400,
            f"{source}在过去1小时内连续失败{failures}次，已切换到后备源")
```

委托人通过调用`analyst_get_system_stats`即可查看系统健康状态。如果Claude在投研报告中调用了后备数据源，应在报告开头标注数据质量提示。

投研记录留痕（便于回顾分析过哪些标的）：

```python
```

```python
# shared/stats.py 中增加
async def record_research(redis, symbol, market, framework, tools_called):
    """记录每次投研的标的和使用的工具"""
    key = f"research_log:{datetime.now().strftime('%Y-%m-%d')}"
    await redis.rpush(key, json.dumps({
        "time": datetime.now().isoformat(),
        "symbol": symbol, "market": market,
        "framework": framework, "tools": tools_called
    }))
    await redis.expire(key, 86400 * 90)  # 保留90天
```

委托人可通过`analyst_get_system_stats`查看近期分析过的标的列表。

心跳告警（主动发现Server宕机）：

```
复用UptimeRobot（已在SSL监控中推荐）：
  监控类型: HTTP(s)
  URL: https://your-domain.com/us-stocks/mcp
  预期响应码: 405（POST-only端点返回405表示Server在运行）
  检测频率: 每5分钟
  告警: 返回非405或超时 → 邮件/微信通知

只需监控1个Server的URL即可。
如果Nginx或Docker宕机，所有Server都不可达，1个探针就能覆盖。
```

9.9 版本管理与回滚

```
1. Git标签: 每次里程碑验收后打tag
   git tag -a v1.0 -m "里程碑1: 美股+A股+加密"
   git tag -a v2.0 -m "里程碑2: 港股+宏观+估值引擎"
   git tag -a v3.0 -m "里程碑3: 情绪+监管+优化"

2. 回滚命令（出问题时30秒恢复）：
   git checkout v1.0
   docker-compose up -d --build

3. 依赖升级流程（防止breaking change）：
   ① 在开发机上修改requirements.txt中的版本号
   ② 运行全量测试: pytest tests/ -v
   ③ 测试通过后提交并打新tag
   ④ 生产服务器拉取代码，用滚动重启部署（见9.10）
   ⚠️ 绝不在生产服务器上直接pip install --upgrade
```

9.10 日常运维操作

```bash
# === 启动/停止 ===
docker-compose up -d                # 启动全部
docker-compose down                 # 停止全部（⚠️ 会导致全部下线）
docker-compose restart analyst-mcp # 重启单个Server

# === 滚动更新（零停机）===
# 更新shared/adapters后，逐个重启Server而不是全量停机：
for server in us-stocks-mcp cn-stocks-mcp crypto-mcp hk-stocks-mcp \
              macro-mcp sentiment-mcp regulatory-mcp analyst-mcp; do
  docker-compose up -d --build $server
  sleep 10
  echo "✅ $server 已更新"
done

# === 单个Server代码更新（最常见场景）===
# 修改了shared/adapters/fmp.py后（volume挂载模式下）：
docker-compose restart us-stocks-mcp analyst-mcp  # 只重启用到fmp的Server

# === 查看资源使用 ===
docker stats --no-stream            # 查看CPU/内存/网络
docker-compose ps                   # 查看容器状态

# === 清理 ===
docker system prune -f              # 清理无用镜像（节省磁盘）
```

9.11 数据源API变更应对流程

```
当某个工具突然返回空数据或报错时：

1. 查日志定位：
   grep -r "失败\|ERROR" logs/ --include="*.log" | tail -20

2. 确认是哪个数据源：
   检查日志中的 source_name 字段（如"FMP | 失败(403)"）

3. 检查数据源更新：
   - AKShare: https://github.com/akfamily/akshare/releases（函数名变更）
   - yfinance: https://github.com/ranaroussi/yfinance/issues（Yahoo API变更）
```

```
   - CoinGecko: https://status.coingecko.com（服务状态）
   - FMP/Finnhub: 检查API Key是否过期或额度变更

4. 本地测试新版本:
   pip install akshare==新版本
   pytest tests/unit/ -k "akshare" -v

5. 更新requirements.txt并部署:
   git commit → git tag → 滚动重启（见9.10）

6. 验证:
   调用 analyst_get_system_stats 确认失败率恢复正常
```

阶段一（第1-2周）：核心三件套

目标: 美股+A股+加密 三个核心Server可用

Week 1: □ 首先开发**shared**/公共层（地基）: - shared/adapters/（FMP+Finnhub+yfinance+SEC EDGAR的adapter） - shared/cache.py（Redis缓存+graceful degradation） - shared/fallback.py（故障切换+告警阈值） - shared/config.py（统一读.env配置） - shared/validators.py + shared/formatters.py - shared/quota.py + shared/stats.py □ 搭建项目骨架（Docker Compose + Redis + Nginx + Dockerfile模板） □ 开发us-stocks-mcp（16个工具，基于shared/adapters调用） □ 部署到云服务器，验证Claude Desktop连接

Week 2:
□ 开发cn-stocks-mcp（15个工具）
□ 集成AKShare + Tushare + BaoStock（含BaoStock连接管理）
□ 开发crypto-mcp（15个工具）
□ 集成DeFiLlama全套API + CoinGecko + CCXT
□ 代币符号消歧义逻辑
□ 全部部署，用BTGO/600519/BTC做集成测试

里程碑1验收标准: ✓ Claude Desktop能通过stdio成功连接3个MCP Server ✓ Claude Desktop能通过HTTPS成功连接3个MCP Server ✓ 输入"BTGO"能获取完整财务三表+分析师评级+股价（返回格式正确，含货币标注） ✓ 输入"600519"能获取茅台完整A股数据（代码格式自动转换正常） ✓ 输入"BTC"能获取比特币价格+DeFiLlama TVL数据 ✓ 手动禁用FMP API后，自动切换到yfinance获取财务数据 ✓ 同一请求第二次调用从Redis缓存返回（< 200ms） ✓ Docker健康检查（TCP端口检测）通过，8个容器全部running ✓ 日志正常记录到文件（含轮转）

阶段二（第3-4周）：补充Server

Week 3:
□ 开发hk-stocks-mcp（10个工具）

□ 开发macro-mcp（13个工具）
□ 集成FRED API + AKShare宏观模块

Week 4:
□ 开发analyst-mcp（10个工具，含估值引擎+系统监控）
□ 重点开发analyst_run_multi_valuation自动估值引擎
□ 全面集成测试

里程碑2验收标准: ✓ 输入"9988"能获取阿里巴巴港股数据（代码格式正确） ✓ analyst_run_multi_valuation能自动输出5+种估值交叉验证表格 ✓ macro_get_us_gdp / macro_get_china_cpi 等宏观数据正常获取 ✓ 所有6个Server同时运行24小时无崩溃 ✓ API额度管理有效（模拟超额时自动切换后备源）

阶段三（第5-6周）: 增强与优化

Week 5:
□ 开发sentiment-mcp（8个工具）
□ 开发regulatory-mcp（6个工具）

Week 6:
□ 性能优化
□ 安全审查（Bearer Token认证、HTTPS、日志脱敏）
□ 编写运维文档
□ 编写测试用例

里程碑3（最终）验收标准: ✓ 8个Server全部上线并稳定运行 ✓ 完整数据获取链路: 输入代码到全部数据就位 < 90秒（缓存未命中）/ < 15秒（缓存命中） ✓ Nginx Bearer Token认证有效（无Token返回401） ✓ 日志可追踪、可轮转 ✓ 运维文档完整

---

11. 测试策略

11.1 单元测试

每个工具至少1个单元测试，使用mock模拟外部API：

```python
```

```python
# tests/unit/test_us_stocks.py
import pytest
from unittest.mock import patch

@pytest.mark.asyncio
async def test_us_get_stock_price_fmp():
    """测试FMP数据源正常返回"""
    with patch("shared.adapters.fmp.get_quote") as mock_fmp:
        mock_fmp.return_value = {
            "symbol": "BTGO", "price": 7.67, "marketCap": 886000000
        }
        result = await us_get_stock_price("BTGO")
        assert "BTGO" in result
        assert "$7.67" in result

@pytest.mark.asyncio
async def test_us_get_stock_price_fallback():
    """测试FMP失败后自动切换到yfinance"""
    with patch("shared.adapters.fmp.get_quote") as mock_fmp:
        mock_fmp.side_effect = Exception("API Error")
        with patch("shared.adapters.yahoo.get_quote") as mock_yf:
            mock_yf.return_value = {"shortName": "BitGo", "currentPrice": 7.67}
            result = await us_get_stock_price("BTGO")
            assert "BitGo" in result
```

11.2 集成测试

每个Server至少1个集成测试（调用真实API验证返回格式）：

```python
# tests/integration/test_us_stocks_live.py
@pytest.mark.integration
@pytest.mark.asyncio
async def test_us_get_income_statement_live():
    """集成测试：调用真实FMP API获取AAPL财务报表"""
    result = await us_get_income_statement("AAPL", period="annual", limit=1)
    assert "总收入" in result or "Revenue" in result
    assert "$" in result
    assert "[数据获取失败]" not in result
```

11.3 缓存共享测试（关键架构验证）

```python
# tests/integration/test_cache_sharing.py
@pytest.mark.integration
@pytest.mark.asyncio
async def test_cache_key_consistency_across_servers():
    """验证：不同Server调用同一adapter生成的cache_key完全相同"""
    from shared.cache import cache_key

    # us-stocks-mcp视角
    key1 = cache_key("fmp", "income_statement", "AAPL", period="annual", limit=3)
    # analyst-mcp视角（相同adapter、相同参数）
    key2 = cache_key("fmp", "income_statement", "AAPL", period="annual", limit=3)

    assert key1 == key2, f"跨Server缓存Key不一致：{key1} vs {key2}"
    assert key1.startswith("fmp:"), f"Key应以数据源名开头，实际：{key1}"

@pytest.mark.integration
@pytest.mark.asyncio
async def test_cache_sharing_read_write(redis_client):
    """验证：一个Server写入的缓存，另一个Server能读到"""
    from shared.cache import cache_key, get_cached_or_fetch

    key = cache_key("fmp", "income_statement", "AAPL", period="annual", limit=3)

    # 模拟us-stocks-mcp写入缓存
    await redis_client.setex(key, 3600, "cached_financial_data")

    # 模拟analyst-mcp读取缓存
    cached = await redis_client.get(key)
    assert cached is not None
    assert cached.decode() == "cached_financial_data"
```

11.4 端到端测试（E2E）

call_tool的实现方式（E2E测试基础设施）：

```python
```

```python
# tests/e2e/conftest.py
import importlib

async def call_tool(tool_name: str, params: dict) -> str:
    """
    E2E测试直接import工具函数调用，不通过MCP协议。
    测试的是工具逻辑+adapter+缓存的完整链路。
    """
    # 根据工具名前缀确定所在Server
    prefix_map = {
        "us_": "us_stocks_mcp.tools",
        "cn_": "cn_stocks_mcp.tools",
        "hk_": "hk_stocks_mcp.tools",
        "crypto_": "crypto_mcp.tools",
        "macro_": "macro_mcp.tools",
        "sentiment_": "sentiment_mcp.tools",
        "reg_": "regulatory_mcp.tools",
        "analyst_": "analyst_mcp.tools",
    }
    for prefix, module_path in prefix_map.items():
        if tool_name.startswith(prefix):
            module = importlib.import_module(module_path)
            func = getattr(module, tool_name)
            return await func(**params)
    raise ValueError(f"Unknown tool: {tool_name}")
```

python

```python
# tests/e2e/test_full_research.py
@pytest.mark.e2e
@pytest.mark.asyncio
async def test_full_growth_research_aapl():
    """
    端到端测试：模拟一次完整的AAPL美股投研。
    依次调用14节框架所需的全部工具，验证数据完整性和总延迟。
    测试标的：AAPL（覆盖率最高，所有数据源都有数据）
    """
    import time
    start = time.time()
    results = {}
    total_chars = 0

    # 调用14节框架所需的核心工具
    tools_to_call = [
        ("us_get_company_profile", {"symbol": "AAPL"}),
        ("us_get_stock_price", {"symbol": "AAPL"}),
        ("us_get_income_statement", {"symbol": "AAPL", "period": "annual", "limit":
        ("us_get_balance_sheet", {"symbol": "AAPL", "period": "annual", "limit": 3})
        ("us_get_cashflow", {"symbol": "AAPL", "period": "annual", "limit": 3}),
        ("us_get_ratios", {"symbol": "AAPL"}),
        ("us_get_insider_trades", {"symbol": "AAPL", "limit": 10}),
        ("us_get_institutional_holders", {"symbol": "AAPL"}),
        ("us_get_peers", {"symbol": "AAPL"}),
        ("analyst_run_multi_valuation", {"symbol": "AAPL", "market": "us", "framewor
    ]

    for tool_name, params in tools_to_call:
        result = await call_tool(tool_name, params)  # 自行实现调用逻辑
        results[tool_name] = result
        total_chars += len(result)

        # 每个工具应返回非空且非错误
        assert len(result) > 0, f"{tool_name} 返回为空"
        assert "[数据获取失败]" not in result, f"{tool_name} 获取失败: {result[:200]}"

    elapsed = time.time() - start

    # 性能断言
    assert elapsed < 120, f"E2E总耗时{elapsed:.0f}s超过120s上限"

    # 数据量断言（控制在Claude上下文安全范围内）
```

```
    assert total_chars < 40000, f"总返回{total_chars}字符超过40,000上限"

    # multi_valuation应返回≥5种有效方法
    mv_result = results["analyst_run_multi_valuation"]
    assert "有效方法" in mv_result
    # 提取有效方法数
    import re
    match = re.search(r"有效方法: (\d+)/12", mv_result)
    assert match and int(match.group(1)) >= 5, "有效估值方法不足5种"

    print(f"✅ E2E测试通过: {elapsed:.1f}s, {total_chars}字符, {len(results)}个工具")

@pytest.mark.e2e
@pytest.mark.asyncio
async def test_full_growth_research_600519():
    """E2E测试: A股路径 (茅台), 验证代码格式自动转换"""
    results = {}
    tools = [
        ("cn_get_company_profile", {"symbol": "600519"}),
        ("cn_get_stock_price", {"symbol": "600519"}),
        ("cn_get_income_statement", {"symbol": "600519", "period": "annual"}),
        ("cn_get_ratios", {"symbol": "600519"}),
        ("cn_get_shareholders", {"symbol": "600519"}),
    ]
    for name, params in tools:
        result = await call_tool(name, params)
        assert "[数据获取失败]" not in result, f"{name} 失败"
        assert len(result) > 0
        results[name] = result
    print(f"✅ A股E2E通过: {len(results)}个工具")

@pytest.mark.e2e
@pytest.mark.asyncio
async def test_full_web3_research_uniswap():
    """E2E测试: 加密路径 (Uniswap), 验证symbol消歧义+protocol_name"""
    results = {}
    tools = [
        ("crypto_get_token_price", {"symbol": "UNI"}),           # symbol类
        ("crypto_get_market_data", {"symbol": "UNI"}),           # symbol类
        ("crypto_get_protocol_tvl", {"protocol_name": "uniswap"}),  # protocol_name类
        ("crypto_get_protocol_revenue", {"protocol_name": "uniswap"}),
        ("crypto_get_sustainability_ratio", {"protocol_name": "uniswap"}),
        ("analyst_run_multi_valuation", {"symbol": "UNI", "market": "crypto", "frame
    ]
```

```python
    for name, params in tools:
        result = await call_tool(name, params)
        assert "[数据获取失败]" not in result, f"{name} 失败"
        results[name] = result
    # web3估值应返回有效方法
    assert "有效方法" in results["analyst_run_multi_valuation"]
    print(f"✅ 加密E2E通过: {len(results)}个工具")

@pytest.mark.e2e
@pytest.mark.asyncio
async def test_cache_speedup():
    """验证缓存命中时性能提升"""
    import time

    # 第一次调用（缓存未命中）
    start1 = time.time()
    await call_tool("us_get_ratios", {"symbol": "AAPL"})
    elapsed1 = time.time() - start1

    # 第二次调用（缓存命中）
    start2 = time.time()
    await call_tool("us_get_ratios", {"symbol": "AAPL"})
    elapsed2 = time.time() - start2

    # 缓存命中应快10倍以上
    assert elapsed2 < elapsed1 / 5, f"缓存加速不足: 首次{elapsed1:.2f}s, 缓存{elapsed2:
```

11.5 负向测试（边界条件与异常场景）

```python
```

```python
# tests/unit/test_negative_cases.py

@pytest.mark.asyncio
async def test_invalid_symbol():
    """无效代码应返回统一错误信息，不崩溃"""
    result = await call_tool("us_get_stock_price", {"symbol": "XYZABC"})
    assert "[标的不存在]" in result or "[数据获取失败]" in result

@pytest.mark.asyncio
async def test_redis_down_graceful():
    """Redis宕机时工具应正常返回数据（无缓存模式）"""
    # 断开Redis后调用
    with patch("shared.cache.redis", side_effect=ConnectionError):
        result = await call_tool("us_get_ratios", {"symbol": "AAPL"})
        assert "[数据获取失败]" not in result  # 应从API直接获取

@pytest.mark.asyncio
async def test_api_key_expired():
    """API Key过期时应自动切换后备源"""
    with patch("shared.adapters.fmp.get_quote", side_effect=Exception("401 Unauthori
        result = await call_tool("us_get_stock_price", {"symbol": "AAPL"})
        assert "AAPL" in result  # 应从yfinance后备获取成功

@pytest.mark.asyncio
async def test_empty_api_response():
    """API返回空数据时应返回友好提示，不崩溃"""
    with patch("shared.adapters.fmp.get_income_statement", return_value={}):
        result = await call_tool("us_get_income_statement", {"symbol": "BTGO"})
        assert "[数据获取失败]" in result or "N/A" in result

@pytest.mark.asyncio
async def test_crypto_wrong_param_type():
    """加密工具传错参数类型（symbol传成protocol_name）应有清晰提示"""
    result = await call_tool("crypto_get_protocol_tvl", {"protocol_name": "UNI"})
    # "UNI"是symbol不是protocol_name，应提示用"uniswap"
    # 或者能智能匹配（UNI → uniswap）

@pytest.mark.asyncio
async def test_cn_stock_outside_trading_hours():
    """A股非交易时段（周末/节假日/夜间）应返回最近交易日收盘数据，不崩溃"""
    result = await call_tool("cn_get_stock_price", {"symbol": "600519"})
    assert "[数据获取失败]" not in result
    assert "最新价" in result or "收盘" in result
```

```
    # AKShare/BaoStock在非交易时段应返回最近交易日数据

@pytest.mark.asyncio
async def test_baostock_login_outside_hours():
    """BaoStock在非交易时段仍能login成功并返回历史数据"""
    result = await call_tool("cn_get_price_history", {
        "symbol": "600519", "start_date": "2025-01-01", "end_date": "2025-12-31"
    })
    assert "[数据获取失败]" not in result
```

11.6 测试运行

```bash
# 单元测试（不需要网络，快速）
pytest tests/unit/ -v

# 集成测试（需要网络和API Key + Redis）
pytest tests/integration/ -v --timeout=30

# 端到端测试（需要完整环境）
pytest tests/e2e/ -v --timeout=120

# 全量测试
pytest tests/ -v --timeout=120
```

---

12. 验收标准（最终交付物）

12.1 代码交付

□ GitHub Private Repo，包含完整代码和README
□ 8个MCP Server源代码（按目录分隔）
□ Dockerfile（每个Server一个）
□ docker-compose.yml
□ nginx.conf
□ .env.example（不含真实Key）
□ requirements.txt（每个Server独立，所有版本pin死）
□ Claude Desktop配置模板（stdio版 + HTTP版）

12.2 功能验收

□ 美股: BTGO/AAPL/COIN，16个工具全部返回正确格式数据 □ A股: 600519/000001/300750，16个

工具全部返回正确格式数据 □ 港股: 9988/0700/1810，10个工具全部返回正确格式数据 □ 加密: BTC/ETH/SOL + "uniswap"/"aave"等协议名，17个工具正常 □ 宏观: 中美GDP/CPI/利率/PMI等正常获取 □ 估值: analyst_run_multi_valuation三种framework各输出12种方法交叉验证表 □ 故障切换: 手动禁用主数据源后自动切换 □ 缓存: 同请求第二次 < 200ms □ 认证: 无Bearer Token返回401 □ 代码转换: A股000001/港股9988/加密BTC自动格式化 □ **标的不存在处理**: 输入"XYZABC"等无效代码，返回统一错误提示而非崩溃 □ **symbol vs protocol_name**: 加密工具传错参数类型时返回清晰提示

12.3 性能要求

□ 单工具调用 < 3秒（缓存未命中，analyst_run_multi_valuation除外）
□ analyst_run_multi_valuation < 30秒（缓存未命中）
□ 缓存命中 < 200ms
□ 8个Server同时运行: CPU < 60%, 内存 < 6GB
□ 连续运行7天无崩溃

12.4 测试交付

□ 每个工具至少1个单元测试
□ 每个Server至少1个集成测试
□ 缓存共享跨Server测试通过
□ 3条E2E路径全部通过（美股AAPL + A股600519 + 加密UNI/uniswap）
□ pytest运行通过率 > 95%

12.5 文档交付

□ 系统架构文档（含部署图）
□ 每个MCP Server工具说明
□ API密钥注册指南（步骤+截图）
□ 部署和启动指南（从零到运行）
□ Claude Desktop配置指南
□ 常见问题排查指南
□ 数据源额度监控方法

12.6 效果验收（系统交付后第一周）

委托人使用系统完成2次真实投研（1次美股/A股 + 1次加密），对比以下指标:

指标	系统前（纯手动）	系统后（MCP+搜索）	目标
单标的投研总时长	3-5小时	30-60分钟	减少70%+
估值方法数量	3-5种（手动算）	12种（自动）	覆盖率100%
数据维度完整度	40-60%	90%+	提升至90%+
每天可分析标的数	0.5-1个	2-3个	提升3-5倍

Research模式输出质量检查（委托人逐项确认）：

□ 14节/8步/15节的每个子问题都有具体数据回答（不是"需要进一步研究"）
□ 12种估值方法的表格完整展示（含N/A原因和部分失败标注）
□ 引用了MCP工具返回的定量数据（不是全靠web搜索的定性描述）
□ 币种标注正确且估值计算无币种错误（特别检查PDD/9988等跨境标的）
□ 投后检查三原则有明确的"是/否"结论
□ 催化剂时间表有具体日期和事件
□ 全文为中文撰写，英文术语有括号标注

---

13. 参考资源

MCP开发

资源	链接
MCP协议规范	https://spec.modelcontextprotocol.io
Python SDK	https://github.com/modelcontextprotocol/python-sdk
官方教程	https://modelcontextprotocol.io/quickstart/server
官方Server示例	https://github.com/modelcontextprotocol/servers

数据源API文档

数据源	文档链接
FMP	https://site.financialmodelingprep.com/developer/docs
Finnhub	https://finnhub.io/docs/api

数据源	文档链接
SEC EDGAR	https://www.sec.gov/search-filings/edgar-application-programming-interfaces
FRED	https://fred.stlouisfed.org/docs/api/fred/
AKShare	https://akshare.akfamily.xyz/data/stock/stock.html
Tushare	https://tushare.pro/document/2
BaoStock	http://baostock.com/baostock/index.php
DeFiLlama	https://defillama.com/docs/api
CoinGecko (Demo)	https://docs.coingecko.com/v3.0.1/reference/introduction
CoinMarketCap	https://coinmarketcap.com/api/documentation/v1/
Etherscan	https://docs.etherscan.io/
CCXT	https://docs.ccxt.com/#/
Snapshot (GraphQL)	https://docs.snapshot.org/graphql-api
Alpha Vantage	https://www.alphavantage.co/documentation/

可参考的开源项目

项目	链接	说明
stock-mcp	https://github.com/huweihua123/stock-mcp	最完整的金融MCP Server参考
stockdata-mcp	https://github.com/xj-bear/stockdata-mcp	轻量AKShare MCP Server

---

14. 沟通与交付流程

14.1 沟通

- 每周末提交周进度报告（完成/问题/下周计划）
- 技术选型变更及时沟通
- 每个里程碑完成后安排验收

14.2 交付方式

- 代码通过GitHub Private Repo交付
- 部署到我指定的云服务器（我提供服务器SSH访问权限）
- 或者你部署好后提供MCP Server的HTTPS URL和Bearer Token

14.3 维护期

- 交付后30天免费修复Bug
- 数据源API变更导致的适配器更新不在免费维护范围内
- 如需长期维护，另行约定

---

15. Claude调用指南（委托人使用须知）

本节面向委托人（而非开发者），说明系统建成后的使用方式。
开发者也必须阅读本节——它影响工具description的写法和返回数据的精简策略。

⚠️ **本系统的主要使用模式：Claude.ai的Research功能 + Opus 4.6模型 + MCP Server。** Research模式下Claude会自主规划研究路径、自主调用工具和搜索，与常规对话模式有根本性差异。

15.1 Server连接策略

Opus 4.6 + Research模式（推荐）

```
建议同时连接全部8个MCP Server。

原因：
  1. Opus 4.6的工具选择能力远强于Sonnet/Haiku，90个工具不构成负担
  2. 90个工具Schema约30K tokens，Opus的200K上下文完全够用
  3. Research模式中途无法切换Server连接——必须事先连接所有可能用到的Server
  4. 你不确定Research过程中Opus会决定调用哪些工具（它可能自主决定查宏观数据或监管文件）
```

Sonnet/Haiku + 常规对话模式（备用）

```
如果使用非Research模式或较弱的模型，建议按标的类型只连4个Server：
  美股：us-stocks-mcp + analyst-mcp + macro-mcp + sentiment-mcp
  A股：cn-stocks-mcp + analyst-mcp + macro-mcp + sentiment-mcp
  港股：hk-stocks-mcp + analyst-mcp + macro-mcp + sentiment-mcp
  加密：crypto-mcp + analyst-mcp + sentiment-mcp
```

15.2 Research模式投研提示词

核心原则：给目标和标准，不给步骤

Research模式下Claude自主规划研究路径。**不要给逐步指令**（"第一步调xx，第二步调yy"），那会束缚Opus的自主深挖能力。正确的做法是给它：

```
✅ 研究目标（要回答什么问题）
✅ 判断标准（怎么判断好坏）
✅ 数据分工（MCP工具 vs web搜索各负责什么）
✅ 输出模板（必须按什么结构输出）

❌ 不要给逐步操作清单
❌ 不要指定调用顺序
```

⚠️ **语言要求：** 无论研究过程中使用了多少英文数据源和英文网页，最终报告**必须全文使用中文撰写**。英文术语首次出现时用括号标注英文原文（如"市盈率(PE)"），后续使用中文。在每个提示词模板末尾都已包含此要求。

数据分工原则（MCP vs web搜索）

Research模式下Claude会同时使用MCP工具和web搜索两个数据通道。必须在提示词中明确分工，否则Claude可能用web搜索重复获取MCP已有的数据，或完全忽略MCP工具。

```
定量数据 → 优先用MCP工具（比web搜索更精确更结构化）：
  财务三表、估值比率、股价行情、分析师评级、内部人交易、
  12种估值交叉验证、恐惧贪婪指数、北向/南向资金、TVL/协议收入等

定性信息 → 用web搜索（MCP无法获取）：
  创始人背景与访谈、管理层风格评价、企业文化分析、
  产品体验评价、竞对深度对比、行业趋势报告、
  监管政策解读、催化剂事件追踪、社区舆情分析

⚠️ 不要用web搜索获取财务数据——MCP工具的数据更准确更完整
⚠️ 必须调用analyst_run_multi_valuation——这是web搜索无法替代的12种估值自动计算
```

框架选择标准

```
growth（成长性公司14节）：上市<10年、营收增速>20%、尚未盈利或刚开始盈利、行业处于成长期
mature（成熟公司8步）：   上市>10年、有稳定盈利历史、行业地位确立、当前因危机被错杀
web3（加密项目15节）：    加密代币/DeFi协议/链上项目
不确定时：先用growth分析，再切换mature看评分结果，取更适合的那套
```

提示词模板①：成长性公司14节框架（Research模式）

```
研究目标：对 [代码]（[市场]）执行成长性公司14节危机投资框架完整分析。

核心公式：超额收益 = 发生可逆危机的好公司 × 被情绪错杀的极低价格 × 市场空间极大的增长潜力 × 耐心等待时间的平方

需要回答的核心问题：
  - 这家公司是否业务简单易懂且有深厚护城河？
  - 管理层是否诚实理性？危机期间是增持还是减持？
  - 赚钱能力和管钱能力在同行中处于什么水平？
  - 财务是否健康——自由现金流、负债率、营收增速？
  - 三年内被颠覆的风险有多大？
  - 用12种估值方法交叉验证，当前股价是否低于内在价值5折？
  - 催化剂信号是否清晰可见？向下风险有限，向上收益无限？

数据获取分工：
  定量数据请优先使用MCP工具（[us/cn/hk]_get_*系列 + analyst_run_multi_valuation）
  定性信息请用web搜索（创始人背景、竞对分析、行业趋势、产品体验、社区评价）
  估值必须调用：analyst_run_multi_valuation(symbol='[代码]', market='[us/cn/hk]', framework='growth')

输出格式（必须严格遵守以下结构，逐节分析，不可自行重组）：
  标题：致投委会投资备忘录 — [公司名]（[代码]）
  一、标的介绍（1.战略定位 2.痛点场景 3.需求验证）
  二、事行不行（1.市场空间 2.增长空间 3.市占率）
  三、人行不行（1.创始人 2.高管 3.团队文化）
  四、业务行不行（1.产品定位 2.盈利模式 3.规模增速）
  五、行业价值大不大（1.降低成本 2.提升效率 3.改善体验）
  六、竞争力强不强（1.垄断潜质 2.生态优势 3.品牌心智）
  七、对手强不强（1.护城河 2.赚钱能力 3.管理团队）
  八、赚钱能力行不行（1.ROIC 2.毛利率 3.成本控制）
  九、管钱能力行不行（1.利润留存 2.所有者收益 3.并购回购）
  十、财务健不健康（1.自由现金流 2.负债率 3.营收增速）
  十一、被颠覆风险大不大（1.盈利模式 2.技术壁垒 3.产品优势）
  十二、买入时机便不便宜（★12种估值交叉验证表格★）
  十三、卖出时机行不行
  十四、建仓原则符不符合（1.愿意当股东 2.睡得着 3.5年10倍）
  投后检查三原则

报告详略：每个问题都必须有数据支撑，估值部分完整展示12种方法表格。预期8,000-15,000字。
```

提示词模板②：成熟公司8步框架（Research模式）

研究目标：对 [代码]（[市场]）执行成熟公司8步危机投资框架完整分析。

核心公式：超额收益 = 发生可逆危机的好公司 × 被情绪错杀的极低价格 × 市场空间极大的增长潜力 × 耐心等待时间的平方

需要回答的核心问题：
- 这家公司遇到了什么危机？能否自救/断臂/他救？（检测危机三步法）
- 人行不行？（10个维度，每题1分，≥6分过关）
- 事行不行？（10个维度，每题1分，≥6分过关）
- 果行不行？（10个财务指标历史行业排名，每题1分，≥6分过关）
- 以上三大维度至少2个过关才能继续
- 用12种估值方法确认是否低估（10种≥3种确认低估即过关）
- 股价是否低于内在价值5折？催化剂是否可见？

数据获取分工：
定量数据请优先使用MCP工具：
· [us/cn/hk]_get_crisis_signals — 检测危机核心数据
· [us/cn/hk]_get_insider_trades — 管理层增减持行为（人行不行Q7）
· analyst_get_historical_ranking_batch — 果行不行10题评分（一次获取全部10项指标排名）
· analyst_run_multi_valuation(framework='mature') — 12种估值交叉验证
定性信息请用web搜索（创始人背景、企业文化、并购历史、催化剂）

输出格式（必须严格遵守，含评分体系）：
标题：致投委会投资备忘录 — [公司名]（[代码]）
一、标的介绍（1.战略定位 2.痛点场景 3.需求验证）
二、检测危机（1.自救求生 2.断臂求生 3.他救求生 → 评分：?/1分，≥1分过关）
三、判断机会
人行不行（10题逐题分析+打分 → 总分?/10分，≥6分过关?）
事行不行（10题逐题分析+打分 → 总分?/10分，≥6分过关?）
果行不行（10项指标历史排名表 → 总分?/10分，≥6分过关?）
三大维度汇总：?/3分（≥2分过关?）
四、确认机会（★12种估值方法检查低估，≥3种确认即过关★）
五、买入时机（1.股价<内在价值5折？ 2.下有限上无限？ 3.催化剂?）
六、卖出时机
七、被颠覆风险大不大
八、建仓原则符不符合（1.愿意当股东 2.睡得着 3.3年5倍）
投后检查三原则

报告详略：果行不行部分必须展示10项指标的完整排名表。估值部分完整展示12种方法表格。预期8,000-15,000字。

提示词模板③：Web3项目15节框架（Research模式）

```
研究目标：对 [代码/协议名]（加密）执行Web3项目15节危机投资框架完整分析。

核心公式：超额收益 = 发生可逆危机的好公司 × 被情绪错杀的极低价格 × 市场空间极大的增长潜力 × 耐心等待时间的平方

需要回答的核心问题：
  - 剥离代币激励后，这个协议是否有真实的用户需求?
  - 手续费收入能否覆盖代币通胀补贴? （可持续性比率>1?）
  - 代币经济模型是否有价值捕获能力（回购/销毁/分红）？
  - 用12种加密原生估值方法交叉验证，当前FDV是否被低估?
  - 未来12个月的代币解锁会造成多大稀释?

数据获取分工：
  定量数据请优先使用MCP工具：
    · crypto_get_protocol_tvl + crypto_get_protocol_revenue — TVL和收入
    · crypto_get_sustainability_ratio — 可持续性比率（核心指标）
    · crypto_get_token_economics — 代币经济模型（通胀/销毁/回购）
    · crypto_get_token_unlocks — 解锁计划
    · crypto_get_governance — DAO治理活跃度
    · analyst_run_multi_valuation(framework='web3') — 12种加密估值
  定性信息请用web搜索（白皮书解读、团队背景、社区分析、路线图进展、竞对对比）

输出格式（必须严格遵守，逐节分析）：
  标题：致投委会投资备忘录 — [项目名]（[代码]）
  一、标的介绍　二、事行不行　三、人行不行　四、业务行不行
  五、行业价值　六、创新能力　七、竞争力　八、生态繁荣度（★可持续性比率★）
  九、经济模型（★代币激励/回购/销毁/解锁★）　十、财务健康
  十一、对手强不强　十二、买入时机（★12种加密估值交叉验证表格★）
  十三、卖出时机　十四、被颠覆风险　十五、建仓原则
  投后检查三原则

报告详略：生态繁荣度和经济模型部分必须有链上数据支撑。估值部分完整展示12种方法表格（用FDV而非股价）。预期8,000-15,000字。
```

提示词模板④：A股标的（Research模式，growth框架）

```
研究目标：对 [6位代码]（A股）执行成长性公司14节危机投资框架完整分析。

核心公式：超额收益 = 发生可逆危机的好公司 × 被情绪错杀的极低价格 × 市场空间极大的增长潜力 × 耐心等待时间的平方
```

```
需要回答的核心问题：
  - 这家公司是否业务简单易懂且有深厚护城河？
  - 管理层是否诚实理性？危机期间是增持还是减持？
  - 赚钱能力和管钱能力在同行中处于什么水平？
  - 财务是否健康——自由现金流、负债率、营收增速？
  - 用12种估值方法交叉验证，当前股价是否低于内在价值5折？

数据获取分工：
  定量数据请优先使用MCP工具（cn_get_*系列）：
    · cn_get_income_statement + cn_get_balance_sheet + cn_get_cashflow（三表）
    · cn_get_ratios（核心指标）
    · cn_get_shareholders + cn_get_insider_trades（股东+高管增减持）
    · cn_get_northbound_flow + cn_get_margin_data（北向资金+融资融券情绪）
    · analyst_run_multi_valuation(symbol='[代码]', market='cn',
framework='growth')
  定性信息请用web搜索（创始人背景、竞对分析、行业趋势、产品体验）

输出格式（必须严格遵守以下结构，逐节分析，不可自行重组）：
  标题：致投委会投资备忘录 — [公司名]（[代码]）
  一、标的介绍（1.战略定位 2.痛点场景 3.需求验证）
  二、事行不行（1.市场空间 2.增长空间 3.市占率）
  三、人行不行（1.创始人 2.高管 3.团队文化）
  四、业务行不行（1.产品定位 2.盈利模式 3.规模增速）
  五、行业价值大不大（1.降低成本 2.提升效率 3.改善体验）
  六、竞争力强不强（1.垄断潜质 2.生态优势 3.品牌心智）
  七、对手强不强（1.护城河 2.赚钱能力 3.管理团队）
  八、赚钱能力行不行（1.ROIC 2.毛利率 3.成本控制）
  九、管钱能力行不行（1.利润留存 2.所有者收益 3.并购回购）
  十、财务健不健康（1.自由现金流 2.负债率 3.营收增速）
  十一、被颠覆风险大不大（1.盈利模式 2.技术壁垒 3.产品优势）
  十二、买入时机便不便宜（★12种估值交叉验证表格★）
  十三、卖出时机行不行
  十四、建仓原则符不符合（1.愿意当股东 2.睡得着 3.5年10倍）
  投后检查三原则

⚠️ A股代码为6位纯数字（如600519、000001），不要用中文名或简称。
⚠️ 无论研究过程中使用了多少英文数据源，最终报告必须全文使用中文撰写。
报告详略：每个问题都必须有数据支撑，估值部分完整展示12种方法表格。预期8,000-15,000字。
```

提示词模板⑤：港股标的（Research模式，mature框架）

```
研究目标：对 [代码]（港股）执行成熟公司8步危机投资框架完整分析。
```

核心公式：超额收益 = 发生可逆危机的好公司 × 被情绪错杀的极低价格 × 市场空间极大的增长潜力 × 耐心等待时间的平方

需要回答的核心问题：
- 这家公司遇到了什么危机？能否自救/断臂/他救？（检测危机三步法）
- 人行不行？（10个维度，每题1分，≥6分过关）
- 事行不行？（10个维度，每题1分，≥6分过关）
- 果行不行？（10个财务指标历史行业排名，每题1分，≥6分过关）
- 以上三大维度至少2个过关才能继续
- 用12种估值方法确认是否低估
- 股价是否低于内在价值5折？催化剂是否可见？

数据获取分工：
定量数据请优先使用MCP工具（hk_get_*系列）：
· hk_get_crisis_signals — 检测危机核心数据
· hk_get_income_statement + hk_get_balance_sheet + hk_get_cashflow（三表）
· hk_get_ratios + hk_get_southbound_flow（财务+南向资金）
· hk_get_insider_trades — 管理层增减持行为
· analyst_get_historical_ranking_batch(symbol='[代码]', market='hk')（果行不行10题）
· analyst_run_multi_valuation(symbol='[代码]', market='hk', framework='mature')
定性信息请用web搜索（创始人背景、企业文化、并购历史、催化剂）

输出格式（必须严格遵守，含评分体系）：
标题：致投委会投资备忘录 — [公司名]（[代码]）
一、标的介绍（1.战略定位 2.痛点场景 3.需求验证）
二、检测危机（1.自救求生 2.断臂求生 3.他救求生 → 评分：?/1分，≥1分过关）
三、判断机会
人行不行（10题逐题分析+打分 → 总分?/10分，≥6分过关?）
事行不行（10题逐题分析+打分 → 总分?/10分，≥6分过关?）
果行不行（10项指标历史排名表 → 总分?/10分，≥6分过关?）
三大维度汇总：?/3分（≥2分过关?）
四、确认机会（★12种估值方法检查低估，≥3种确认即过关★）
五、买入时机（1.股价<内在价值5折？ 2.下有限上无限？ 3.催化剂?）
六、卖出时机
七、被颠覆风险大不大
八、建仓原则符不符合（1.愿意当股东 2.睡得着 3.3年5倍）
投后检查三原则

⚠️ 港股代码为4-5位数字（如9988、0700），不要补零到6位。
⚠️ 无论研究过程中使用了多少英文数据源，最终报告必须全文使用中文撰写。

```
报告详略：果行不行部分必须展示10项指标的完整排名表。估值部分完整展示12种方法表格。预期8,000-15,000字。
```

15.3 Research模式的执行特性

```
Research模式与常规对话模式的关键差异：

1. 自主规划：Claude自己决定搜索什么、调什么工具、按什么顺序
   → 不要给逐步指令，只给目标和标准

2. 多轮执行：Claude会自动进行多轮搜索和工具调用，不会中途截断
   → 不需要发"继续"，让它跑完

3. 执行时长：通常5-15分钟（比常规对话的3-5分钟更长但更深入）
   → 耐心等待，不要中途打断

4. 上下文管理：Research会累积大量搜索结果，上下文压力更大
   → MCP工具返回的数据必须极度精简（见15.4节）

5. 输出格式：Research可能按自己的逻辑组织内容，而非你的框架结构
   → 必须在提示词中用完整的输出模板强制指定结构（已在上方模板中包含）
```

15.4 工具返回数据精简原则

Research模式下精简尤为重要——Research会进行大量web搜索，上下文空间更紧张。 MCP工具返回只应包含**关键数据数值**，不包含解释性文字——解释是Claude的工作。

```
开发者须遵守的返回数据精简原则：

1. 财务报表默认只返回最近3年（需要更多由Claude指定limit参数）
2. 可比公司列表只返回前5家（按市值接近度排序）
3. 新闻只返回最近5条
4. 所有数值预格式化（用M/B后缀，不返回原始大数字如16150000000）
5. 单个工具返回控制在2000字符以内（约800 tokens）
6. 返回纯数据，不返回分析解读文字
   ✅ "PE:15.2 PB:3.1 ROE:22.1% ROIC:18.5% 毛利率:45.2%"
   ❌ "市盈率(PE)为15.2倍，表示投资者愿意为每元收益支付15.2元..."
```

```
上下文空间估算（Research模式）：
  工具定义Schema: ~30,000 tokens（90个工具）
  MCP工具返回: ~12,000 tokens（15个工具 × 800 tokens）
  web搜索返回: ~45,000 tokens（15次搜索 × 3,000 tokens）
  Claude中间思考: ~20,000 tokens
  最终报告: ~10,000 tokens
  ────────────────
  合计: ~117,000 tokens（Opus 200K窗口的58%，安全范围）
```

15.5 工具描述文案编写规则（开发者必读）

工具的description是Claude选择调用哪个工具的唯一依据。 Research模式下Opus的工具选择方式是先规划研究大纲，再批量匹配工具—— 因此description需要同时包含"返回什么数据"和"能回答什么问题"。

编写规则：

```
1. 第一句: 一句话说明功能和返回的数据类型
2. 第二行: 列出具体返回字段
3. 第三行: 能回答什么问题（适配Opus的规划式工具选择）
4. 第四行: 使用场景——什么框架/什么情况下该调用
5. 第五行: 参数格式说明（symbol/market/framework的合法值）
6. 禁止: 不要写"获取数据"、"查询信息"这种泛泛描述
```

对比示例：

```python
```

```python
# ❌ 差的描述
@mcp.tool()
async def us_get_ratios(symbol: str) -> str:
    """获取美股公司的财务比率。"""

# ✅ 好的描述（适配Research模式的Opus规划能力）
@mcp.tool()
async def us_get_ratios(symbol: str) -> str:
    """获取美股公司核心估值和盈利指标：PE/PB/PS/EV-EBITDA/ROE/ROA/ROIC/
    毛利率/净利率/资产负债率/流动比率/速动比率。
    能回答：这家公司赚钱能力强不强？财务是否健康？估值是高还是低？
    symbol为纯大写字母代码（如BTGO、AAPL、PDD）。"""

# ❌ 差的描述
@mcp.tool()
async def analyst_get_historical_ranking_batch(...):
    """获取历史排名数据。"""

# ✅ 好的描述（包含触发条件——告诉Opus什么时候必须调用）
@mcp.tool()
async def analyst_get_historical_ranking_batch(symbol: str, market: str, years: int
    """【执行成熟公司框架时必须调用】一次性获取10个财务指标的5年历史值及行业排名表。
    返回指标：roic/revenue_growth/cash_ratio/cost_ratio/debt_ratio/
    owner_earnings/retained_value/net_income/capex_ratio/roe。
    能回答：公司的经营指标历史上是否处于行业第一梯队？当前恶化是周期性还是结构性？
    这是成熟公司框架'果行不行'10题评分的唯一数据来源——没有此数据无法评分。
    market: 'us'/'cn'/'hk'。"""

# ✅ 好的描述（包含"不可替代"提示——防止Opus用web搜索替代）
@mcp.tool()
async def analyst_run_multi_valuation(symbol: str, market: str, framework: str) -> s
    """【每次投研必须调用，web搜索无法替代】
    按指定框架自动运行12种估值方法并输出交叉验证表。
    framework: 'growth'(成长性12种)/'mature'(成熟12种)/'web3'(加密12种)。
    market: 'us'/'cn'/'hk'/'crypto'。
    返回每种方法的隐含价值、与当前价格偏离、中位数结论、部分失败标注。
    这是整个投研框架中'买入时机'判断的核心——12种估值交叉验证只有此工具能计算。"""

# ✅ 好的描述（加密工具区分symbol和protocol_name）
@mcp.tool()
async def crypto_get_sustainability_ratio(protocol_name: str) -> str:
    """【执行Web3框架时必须调用，web搜索无法替代】
    计算DeFi协议的经济可持续性比率：真实手续费收入/代币激励排放价值。
```

```
    比率>1=协议能自我维持，<1=仍依赖代币补贴。
    能回答：剥离代币激励后，这个协议有真实的用户需求吗？经济模型是否可持续？
    protocol_name是DeFiLlama协议名（如'uniswap'/'aave'），不是代币符号！"""
```

15.6 MCP不可用时的降级方案

如果MCP Server临时无法连接（网络问题/SSL过期/Server崩溃），不意味着当天无法做投研。

```
降级方案：
  1. Claude仍可通过web_search获取大部分数据（速度慢但可用）
  2. 将提示词中的MCP工具说明删除，改为"请通过web搜索获取全部数据"
  3. 估值计算改为让Claude用代码执行或手动Excel计算
  4. 数据覆盖率从90%降至约50-60%，Research时间从10分钟增至20-30分钟

  MCP是加速器，不是唯一路径。系统挂了不影响投研能力的基本面，只影响效率。
```

附录A：各市场精选数据源清单（42个）

精选原则（少则得，多则惑）：

- 免费（或极低成本）且有API或Python库可直接调用
- 每个数据维度只保留1个主源+1个后备，不堆砌
- 网页抓取类全部砍掉——由Claude联网搜索替代
- 每个市场保留1个"万能源"，它一个覆盖60-70%需求
- 去重后共42个独立数据源，需要开发约25个adapter

三层数据架构提醒：

- **MCP工具层（本清单）** → 结构化定量数据，API直接获取
- **Claude联网搜索层** → 行业研报、创始人访谈、产品体验、官方博客、招聘数据、社区讨论等非结构化信息
- **用户上传层** → 白皮书PDF、付费研报、Wind导出等

A.1 美股数据源（15个）

#	数据源	网站	接入方式	免费额度	覆盖维度	优先级
1	**FMP**	financialmodelingprep.com	REST API	250次/天	行情+基本面+治理+分析师+估值	★万能主力
2	**Finnhub**	finnhub.io	REST API	60次/分钟	行情+治理+分析师+情绪+监管	★万能主力
3	**SEC EDGAR**	sec.gov/edgar	REST API	10次/秒	基本面+治理+监管（全部SEC文件）	★核心
4	**FRED**	fred.stlouisfed.org	REST API	无限	宏观（GDP/CPI/利率/就业/VIX）	★核心
5	**yfinance**	—	Python库	无限⚠️	行情+基本面后备（非官方）	后备
6	**Alpha Vantage**	alphavantage.co	REST API + MCP直连	25次/天	行情+技术指标	后备
7	**Marketaux**	marketaux.com	REST API	100次/天	情绪（新闻NLP情绪评分）	补充
8	**Google Trends**	trends.google.com	Python: pytrends	免费	情绪（搜索热度趋势）	补充
9	**CNN Fear & Greed**	cnn.com/markets	网页抓取	免费	情绪（恐惧贪婪指数）	补充
10	**BLS**	bls.gov	REST API	免费	宏观（就业/CPI明细）	补充
11	**World Bank**	data.worldbank.org	REST API	免费	宏观（全球GDP对比）	补充
12	**Congress.gov**	congress.gov	REST API	免费	监管（立法进程追踪）	补充
13	**Quartr**⭐ MCP直连	quartr.com	Claude MCP连接器	免费基础版	分析师（财报电话会原文）	MCP

#	数据源	网站	接入方式	免费额度	覆盖维度	优先级
14	**Aiera** ⭐ MCP直连	aiera.com	Claude MCP连接器	免费基础版	监管（SEC实时推送）	MCP
15	**Twelve Data**	twelvedata.com	REST API	800次/天	行情+技术指标后备	后备

A.2 A股数据源（15个）

#	数据源	网站	接入方式	免费额度	覆盖维度	优先级
1	**AKShare**	akshare.akfamily.xyz	Python库	无限	行情+基本面+治理+情绪+宏观	★万能主力
2	**Tushare Pro**	tushare.pro	Python API	¥500/年	基本面+治理+分析师	★核心
3	**BaoStock**	baostock.com	Python库	无限	行情+基本面后备	★核心
4	**Efinance**	github.com	Python库	无限	行情+情绪（资金流+龙虎榜）	后备
5	聚宽 **JoinQuant**	joinquant.com	Python SDK	50万条/天	估值（PE/PB历史分位）	补充
6	**FRED**	fred.stlouisfed.org	REST API	无限	宏观（美债→A股估值锚）	补充
7	巨潮资讯网	cninfo.com.cn	网页+API	免费	监管（公告原文+问询函）	★核心
8	百度指数	index.baidu.com	网页	免费	情绪（搜索热度）	补充
9	**Google Trends**	trends.google.com	pytrends	免费	情绪（全球视角搜索热度）	补充
10	天眼查	tianyancha.com	网页	基础免费	治理（股权穿透+司法风险）	补充

#	数据源	网站	接入方式	免费额度	覆盖维度	优先级
11	**Ashare**	github.com	Python单文件	免费	行情（实时Tick后备）	后备
12	国家统计局	stats.gov.cn	AKShare集成	免费	宏观（GDP/CPI/PPI）	核心
13	中国人民银行	pbc.gov.cn	AKShare集成	免费	宏观（利率+社融/M2）	核心
14	证监会 **CSRC**	csrc.gov.cn	AKShare集成	免费	监管（行政处罚）	补充
15	上交所/深交所	sse.com.cn/szse.cn	AKShare集成	免费	监管（问询+纪律处分）	补充

注：第12-15号源通过AKShare即可获取，不需要单独写adapter。

A.3 港股数据源（15个）

#	数据源	网站	接入方式	免费额度	覆盖维度	优先级
1	**yfinance (.HK)**	—	Python库	无限⚠️	行情+基本面+分析师	★万能主力
2	**AKShare** 港股模块	akshare.akfamily.xyz	Python库	无限	行情+情绪（南向资金）	★核心
3	**Tushare Pro** 港股	tushare.pro	Python API	¥500/年	治理（港股通持仓）	★核心
4	港交所披露易	hkexnews.hk	网页+RSS	免费	监管+治理（公告+权益披露）	★核心
5	**FRED**	fred.stlouisfed.org	REST API	无限	宏观（美债→港股核心变量）	核心
6	**HKMA** 金管局	hkma.gov.hk	网页	免费	宏观（港元汇率+HIBOR）	补充

#	数据源	网站	接入方式	免费额度	覆盖维度	优先级
7	香港统计处	censtatd.gov.hk	网页	免费	宏观（香港GDP/CPI）	补充
8	**SFC 证监会**	sfc.hk	网页	免费	监管（执法+纪律处分）	补充
9	**FMP**	financialmodelingprep.com	REST API	250次/天	基本面后备（部分港股）	后备
10	**Finnhub**	finnhub.io	REST API	60次/分钟	分析师+情绪后备	后备
11	**Google Trends**	trends.google.com	pytrends	免费	情绪	补充
12	**Stooq (.HK)**	stooq.com	CSV下载	免费	行情（历史日线后备）	后备
13	**World Bank**	data.worldbank.org	REST API	免费	宏观（香港vs全球对比）	补充
14	**Quartr** ⭐ MCP直连	quartr.com	Claude MCP连接器	免费基础版	分析师（部分港股电话会）	MCP
15	**FactSet** ⭐ MCP直连	factset.com	Claude MCP连接器	付费	基本面+分析师（港股覆盖最全）	MCP

A.4 Web3/加密数据源（15个）

#	数据源	网站	接入方式	免费额度	覆盖维度	优先级
1	**DeFiLlama**	defillama.com	REST API（6个子API）	无限	基本面+治理+监管（万能）	★万能主力
2	**CoinGecko**	coingecko.com	REST API	30次/分钟	行情+基本面	★核心
3	**CCXT**	github.com/ccxt	Python库	按交易所	行情（100+交易所）	★核心

#	数据源	网站	接入方式	免费额度	覆盖维度	优先级
4	**CoinMarketCap**	coinmarketcap.com	REST API	10,000次/月	行情+基本面后备	后备
5	**Etherscan**	etherscan.io	REST API	5次/秒	治理+监管（链上验证）	★核心
6	**Snapshot**	snapshot.org	GraphQL API	无限	治理（DAO投票）	★核心
7	**TokenTerminal**	tokenterminal.com	网页+有限API	免费版有限	估值（P/S/P/F倍数）	补充
8	**Coinglass**	coinglass.com	网页+部分API	基础免费	情绪（合约+多空比+爆仓）	补充
9	**Alternative.me**	alternative.me	REST API	免费	情绪（Crypto F&G Index）	补充
10	**CoinDesk**	coindesk.com	网页	免费	行业（新闻+BPI指数）	补充
11	**L2Beat**	l2beat.com	网页	免费	行业（L2 TVL+风险评级）	补充
12	**CryptoCompare**	cryptocompare.com	REST API	100,000次/月	行情后备	后备
13	**LunarCrush** ⭐ MCP直连	lunarcrush.com	Claude MCP连接器	免费	情绪（社交情绪评分）	MCP
14	**Crypto.com** ⭐ MCP直连	crypto.com	Claude MCP连接器	免费	行情（MCP直连价格）	MCP
15	— DeFiLlama子API汇总	—	—	—	见下方	参考

DeFiLlama的6个子API端点（全部免费无限制）：

```
主API（TVL）:     https://api.llama.fi
稳定币API:       https://stablecoins.llama.fi
收益率API:       https://yields.llama.fi
DEX交易量API:    https://api.llama.fi/overview/dexs
手续费/收入API:  https://api.llama.fi/overview/fees
安全事件API:     https://api.llama.fi/hacks
```

A.5 跨市场汇总

去重后的独立数据源总数

类别	数量	包含
美股独有	7	FMP, SEC EDGAR, Marketaux, BLS, Congress.gov, Twelve Data, Aiera
A股独有	9	AKShare, Tushare, BaoStock, Efinance, 聚宽, Ashare, 巨潮, 百度指数, 天眼查
港股独有	5	港交所披露易, HKMA, 香港统计处, SFC, Stooq
加密独有	10	DeFiLlama, CCXT, CoinMarketCap, Etherscan, Snapshot, TokenTerminal, Coinglass, Alternative.me, CoinDesk, CryptoCompare
跨市场共用	8	Finnhub, yfinance, FRED, Alpha Vantage, Google Trends, World Bank, Quartr, FactSet
MCP直连	3	LunarCrush, Crypto.com, (Quartr/Aiera/FactSet已计入上方)
去重总计	**42**	—

每个市场的"万能源"

```
美股: FMP — 一个源覆盖行情+基本面+治理+分析师+估值的60%
A??: AKShare — 一个库覆盖行情+基本面+治理+情绪+宏观的70%
港股: yfinance(.HK) — 唯一免费港股财务三表API
加密: DeFiLlama — 一个源覆盖TVL+收入+手续费+DEX+稳定币+安全事件的60%
```

Claude联网搜索负责的维度（不需要MCP接入）

```
以下信息由Claude通过web_search获取，不需要写adapter：
  行业研究报告（McKinsey/BCG/Bernstein等）
  创始人访谈、高管访谈
  官方网站、官方博客、官方推特
  Youtube产品体验讲解
  路线图文档、技术文档、产品介绍
  招聘数据（LinkedIn/Boss直聘）
  社区关注重点（Reddit/Discord/雪球/Twitter CT）
  做空报告全文（Hindenburg/Muddy Waters等）
  董事会争议治理提案分析
  白皮书（用户上传PDF）
  审计报告（SEC EDGAR下载 / 用户上传）
```

MCP可直连数据源（无需开发，在Claude设置中连接）

#	名称	类型	功能
1	FactSet	付费	机构级财务+分析师共识
2	S&P Global (Kensho)	付费	基本面+行业对标
3	Quartr	免费基础版	财报电话会+管理层PPT
4	Aiera	免费基础版	SEC实时推送+事件日历
5	Crypto.com	免费	加密实时价格
6	LunarCrush	免费	加密社交情绪
7	Alpha Vantage	免费	行情+技术指标

可参考的开源MCP Server项目

#	项目	GitHub	说明
1	stock-mcp	github.com/huweihua123/stock-mcp	最完整参考，已集成AKShare+Tushare+CCXT
2	stockdata-mcp	github.com/xj-bear/stockdata-mcp	轻量AKShare MCP Server

附录B：三套投研框架结构与MCP工具映射

本附录的目的： 让开发者理解每个MCP工具在投研流程中的位置。开发者在编写工具的描述文案（description）时，应参考此映射表，确保Claude在正确的上下文中调用正确的工具。

B.1 成长性公司危机投资投研法（14节）

适用市场：美股、港股、A股

框架节点	分析要点	需要的MCP工具	需要联网搜索补充
一、标的介绍	战略定位+痛点场景+需求验证	[us/cn/hk]_get_company_profile, [us/cn/hk]_get_stock_price	创始人访谈、产品介绍、用户评价
二、事行不行	市场空间+增长+市占率	[us/cn/hk]_get_peers（市占率对比）	行业研报TAM预测（McKinsey/BCG等）
三、人行不行	创始人+高管+文化	[us/cn/hk]_get_insider_trades（管理层行为），us_get_sec_filings（DEF 14A薪酬）	创始人背景、Glassdoor评价、招聘数据
四、业务行不行	产品定位+盈利模式+增速	[us/cn/hk]_get_income_statement（收入增速），[us/cn/hk]_get_ratios	产品体验、客户案例、合作公告
五、行业价值	降低成本+提升效率+改善体验	macro_get_*（行业宏观背景）	行业案例研究、效率对比数据
六、竞争力	垄断潜质+生态优势+品牌心智	[us/cn/hk]_get_peers, analyst_get_peer_valuation	品牌调研、用户心智分析
七、对手强不强	竞对护城河+赚钱能力+管理层	analyst_get_peer_valuation, [us/cn/hk]_get_ratios（竞对的）	竞对产品对比、私有竞对信息
八、赚钱能力	ROIC+毛利率+成本控制	[us/cn/hk]_get_ratios, [us/cn/hk]_get_income_statement, analyst_get_historical_ranking	—
九、管钱能力	利润留存+所有者收益+并购	[us/cn/hk]_get_cashflow, [us/cn/hk]_get_balance_sheet	并购历史及战略分析
十、财务健康	自由现金流+负债率+营收增速	[us/cn/hk]_get_ratios, [us/cn/hk]_get_balance_sheet, [us/cn/hk]_get_cashflow	—

框架节点	分析要点	需要的MCP工具	需要联网搜索补充
十一、被颠覆风险	盈利模式/技术/产品被颠覆可能性	reg_get_*（监管变化），sentiment_get_news_sentiment	技术趋势、竞对动态、监管政策
十二、买入时机★	**10+种估值交叉验证**	**analyst_run_multi_valuation(framework="growth")**，analyst_get_price_targets，sentiment_get_fear_greed	VanEck等独立研究估值
十三、卖出时机	估值过高+盈利确定性丧失+管理层问题	[us/cn/hk]_get_stock_price，analyst_get_valuation_percentile	市场情绪极端信号
十四、建仓原则	愿意当股东+睡得着+5年10倍	综合以上所有工具	综合以上所有定性判断

B.2 成熟公司危机投资投研法（8大步骤）

适用市场：美股、港股、A股

框架步骤	分析要点	需要的MCP工具	需要联网搜索补充
一、标的介绍	一句话定位+痛点+需求验证	[us/cn/hk]_get_company_profile	产品介绍、用户评价
二、检测危机★	自救(降本裁员)+断臂(聚焦主业)+他救(还能融资吗)	**[us/cn/hk]_get_crisis_signals**, [us/cn/hk]_get_cashflow, [us/cn/hk]_get_balance_sheet	裁员新闻、重组公告、融资动态
三、判断机会 — 人行不行（10题≥6分）	使命愿景+企业文化+领导力+并购回购+管理层增减持...	[us/cn/hk]_get_insider_trades (Q7增减持), us_get_sec_filings (薪酬+治理), analyst_get_historical_ranking	创始人访谈、文化评价、并购历史分析
三、判断机会 — 事行不行（10题≥6分）	垄断权+市占率+定价权+创新力+增长潜力...	[us/cn/hk]_get_peers (市占率Q2), [us/cn/hk]_get_ratios (定价权Q3→毛利率)	专利数据、渠道分析、品牌调研
三、判断机会 — 果行不行（10题≥6分）	ROIC/收入增速/净现比/成本占比/负债率/ROE等10项历史排名	**analyst_get_historical_ranking** (10题全靠此工具), [us/cn/hk]_get_ratios	—
四、确认机会★	10种估值方法检查低估	**analyst_run_multi_valuation(framework="mature")**	—
五、买入时机	内在价值5折+向下有限向上无限+催化剂	analyst_get_price_targets, sentiment_get_fear_greed	催化剂事件追踪
六、卖出时机	股价超过内在价值+盈利确定性丧失+管理层道德问题	analyst_get_valuation_percentile, [us/cn/hk]_get_insider_trades	市场狂热信号
七、被颠覆风险	盈利模式/技术/产品三年内被颠	reg_get_*, sentiment_get_news_sentiment	技术趋势、竞对

框架步骤	分析要点	需要的MCP工具	需要联网搜索补充
	覆可能性		动态
八、建仓原则	愿意当股东+睡得着+3年5倍	综合以上	综合以上

关键工具依赖：

- `[us/cn/hk]_get_crisis_signals` 是步骤二的核心——没有这个工具，"检测危机"三步法的数据全靠联网搜索拼凑
- `analyst_get_historical_ranking` 是步骤三"果行不行"的核心——10题中每题都需要"历史行业排名+当前偏离分析"
- `analyst_run_multi_valuation(framework="mature")` 是步骤四的核心——10种估值方法自动计算

B.3 Web3项目危机投资投研法（15节）

适用市场：加密货币（DeFi/基础设施/Layer2等）

框架节点	分析要点	需要的MCP工具	需要联网搜索补充
一、标的介绍	战略定位+痛点+需求验证	crypto_get_market_data, crypto_get_token_price	白皮书、官网、产品文档
二、事行不行	赛道TAM+增长空间+市占率	crypto_get_protocol_tvl (TVL市占率), crypto_get_dex_volume (交易量市占率), crypto_get_chain_comparison	行业研报（Messari/Galaxy/Bernstein）
三、人行不行	创始人+团队+使命驱动	crypto_get_governance (DAO治理活跃度间接反映)	创始人访谈、团队LinkedIn、招聘数据
四、业务行不行	产品定位+剥离代币后留存+刚需验证	crypto_get_protocol_revenue (真实收入), **crypto_get_sustainability_ratio** (剥离补贴后的真实需求)	产品体验、用户反馈、Youtube演示
五、行业价值	降低成本+提升效率+改善体验	crypto_get_chain_comparison (效率对比)	传统行业vs链上效率对比案例
六、创新能力	技术架构+业务逻辑+共识机制创新	crypto_get_security_audit (技术质量间接指标)	技术文档、GitHub代码审查、审计报告
七、竞争力	垄断潜质+生态优势+宗教共识	crypto_get_protocol_tvl (vs竞品), crypto_get_market_data (市值排名)	社区文化分析、品牌认知度
八、生态繁荣度★	社区活跃+TVL增长+手续费能否覆	crypto_get_protocol_tvl, crypto_get_protocol_revenue, **crypto_get_sustainability_ratio**	社区活跃度（Discord/Twitter）

框架节点	分析要点	需要的MCP工具	需要联网搜索补充
	盖代币通胀		
九、经济模型★	激励兼容+价值捕获(回购/销毁)+解锁对齐	**crypto_get_token_economics**(回购/销毁/通胀), crypto_get_token_unlocks(解锁计划)	代币经济学白皮书分析
十、财务健康	DAO金库+有机收入+流动性	crypto_get_protocol_revenue(收入), crypto_get_protocol_tvl(流动性)	DAO国库地址链上查询
十一、对手强不强	竞对护城河+赚钱能力+团队	crypto_get_chain_comparison(链级对比), crypto_get_protocol_revenue(竞对收入)	竞对团队背景、融资历史
十二、买入时机★	10+种加密原生估值交叉验证	**analyst_run_multi_valuation(framework="web3")**, crypto_get_market_data	—
十三、卖出时机	币价超内在价值+叙事转移+范式转移	crypto_get_token_price, sentiment_get_fear_greed_crypto	市场狂热信号
十四、被颠覆风险	盈利模式/技术/产品被颠覆可能性	crypto_get_security_audit, reg_get_crypto_hacks	新协议技术对比、监管政策变化
十五、	愿意当股东+睡得	综合以上	综合以上

框架节点	分析要点	需要的MCP工具	需要联网搜索补充
建仓原则	着+5年10倍		

关键工具依赖：

- `crypto_get_sustainability_ratio` 是第四节和第八节的核心——判断"剥离代币补贴后是否有真实需求"
- `crypto_get_token_economics` 是第九节的核心——判断代币是否有价值捕获能力
- `analyst_run_multi_valuation(framework="web3")` 是第十二节的核心——12种加密原生估值方法

B.4 工具描述文案编写指南

开发者在编写每个MCP工具的 `description` 字段时，应参考以上映射表，在描述中说明该工具服务于哪个投研场景。这能帮助Claude在正确的上下文中调用正确的工具。

示例：

```python
@mcp.tool()
async def us_get_crisis_signals(symbol: str) -> str:
    """获取美股公司的危机检测数据包，用于判断公司面临的危机是否可逆。

    返回内容包括：最近6个月裁员/重组新闻、管理费用(SG&A)变化趋势、
    最近融资/发债记录、信用评级变动、现金/短期负债比率、最近资产出售记录。

    使用场景：当一家公司股价暴跌时，用此工具判断其是否具备"自救"（降本裁员）、
    "断臂"（聚焦主业）或"他救"（外部融资）的能力。

    Args:
        symbol: 美股股票代码，如 BTGO，AAPL，COIN
    """
```

```python
```

```python
@mcp.tool()
async def crypto_get_sustainability_ratio(protocol_name: str) -> str:
    """计算DeFi协议的经济可持续性比率：真实手续费收入 / 代币激励排放价值。

    比率 > 1 表示协议能自我维持（手续费收入覆盖代币补贴），
    比率 < 1 表示协议仍依赖代币排放补贴维持TVL。

    使用场景：判断一个DeFi协议剥离代币激励后是否有真实的用户需求和经济模型。

    Args:
        protocol_name: DeFi协议名称，如 "uniswap", "aave", "gmx"
    """
```

附录C：金融术语表

开发者在编写工具描述和返回格式时需要理解的核心术语：

缩写	全称	中文	说明
PE	Price/Earnings	市盈率	股价/每股收益，最常用的估值指标
PB	Price/Book	市净率	股价/每股净资产
PS	Price/Sales	市销率	市值/营业收入
PEG	PE/Growth	市盈增长比	PE除以盈利增长率，<1通常被认为低估
EPS	Earnings Per Share	每股收益	净利润/总股数
BPS	Book Value Per Share	每股净资产	股东权益/总股数
ROE	Return on Equity	净资产收益率	净利润/股东权益
ROA	Return on Assets	总资产收益率	净利润/总资产
ROIC	Return on Invested Capital	投入资本收益率	衡量资本使用效率的核心指标
WACC	Weighted Average Cost of Capital	加权平均资本成本	DCF估值中的折现率
DCF	Discounted Cash Flow	现金流折现	用未来现金流的现值估算企业价值
EV	Enterprise Value	企业价值	市值+净负债，比市值更全面的企业定价
EBITDA	Earnings Before Interest, Tax, D&A	息税折旧摊销前利润	衡量经营盈利的常用指标
FCF	Free Cash Flow	自由现金流	经营现金流-资本开支
NCAV	Net Current Asset Value	净流动资产价值	流动资产-总负债（格雷厄姆经典指标）
FDV	Fully Diluted Valuation	完全稀释估值	加密项目：代币总供应量×当前价格
TVL	Total Value Locked	总锁仓价值	DeFi协议中锁定的资产总值
P/F	Price/Fees	价格/手续费比	FDV/年化手续费收入（加密原生估值指标）
MVRV	Market Value/Realized Value	市值/已实现价值	链上估值指标，>3.5通常高估
ERP	Equity Risk Premium	股权风险溢价	投资者要求的超过无风险利率的回报，通常5-6%